# Machine Intelligence 1 Tutorial

## 12 Efficient inference in Bayesian Networks via: Junction Trees & Message Passing

Fachgebiet Neuronale Informationsverarbeitung (NI)

WiSe 25/26

Efficient Inference in Bayesian Networks

Message Passing

Message Passing Example

References

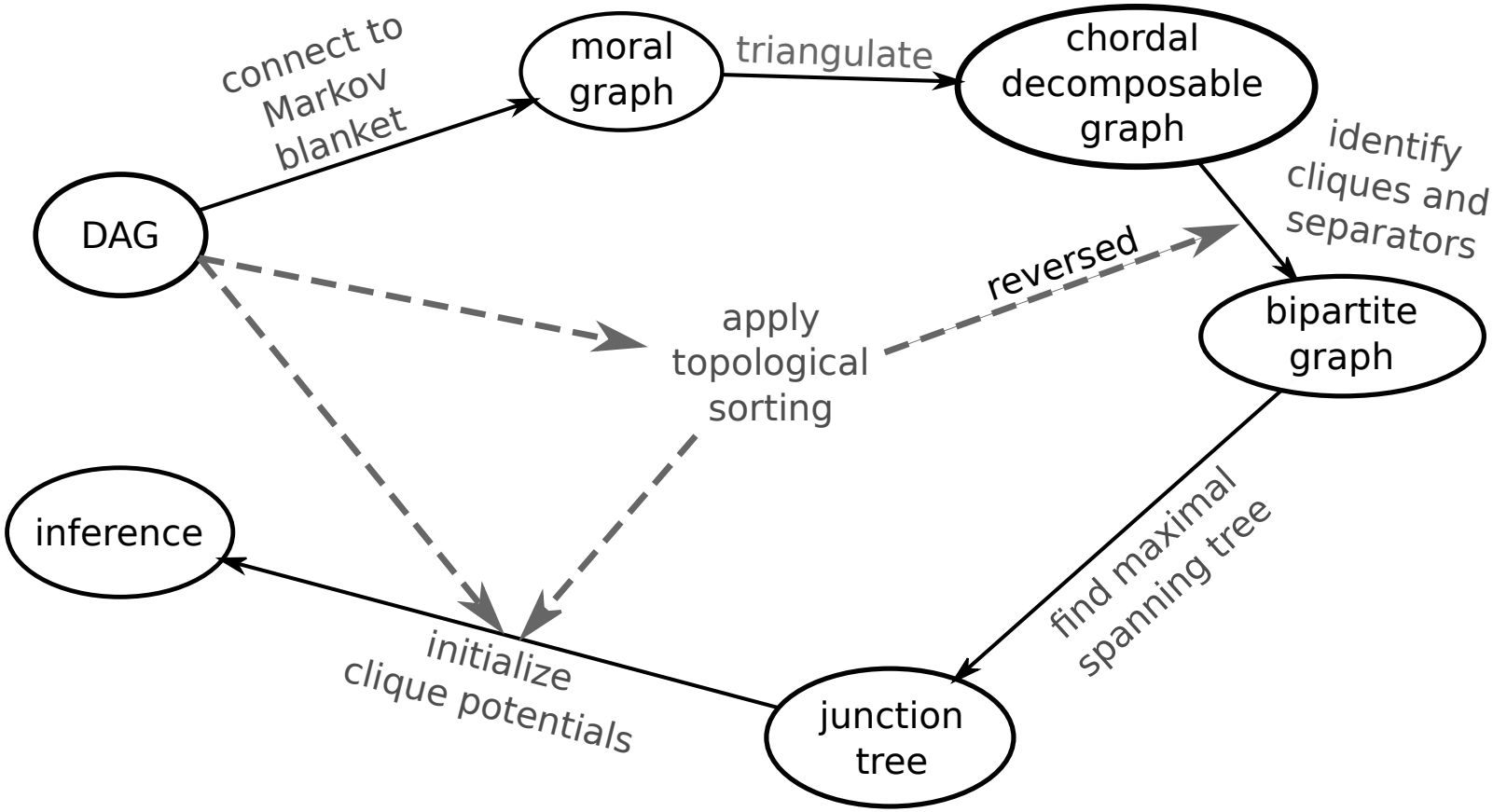
# Efficient Inference in Bayesian Networks

- a compact representation: turning DAG into a Junction tree,
- efficient Inference using message passing

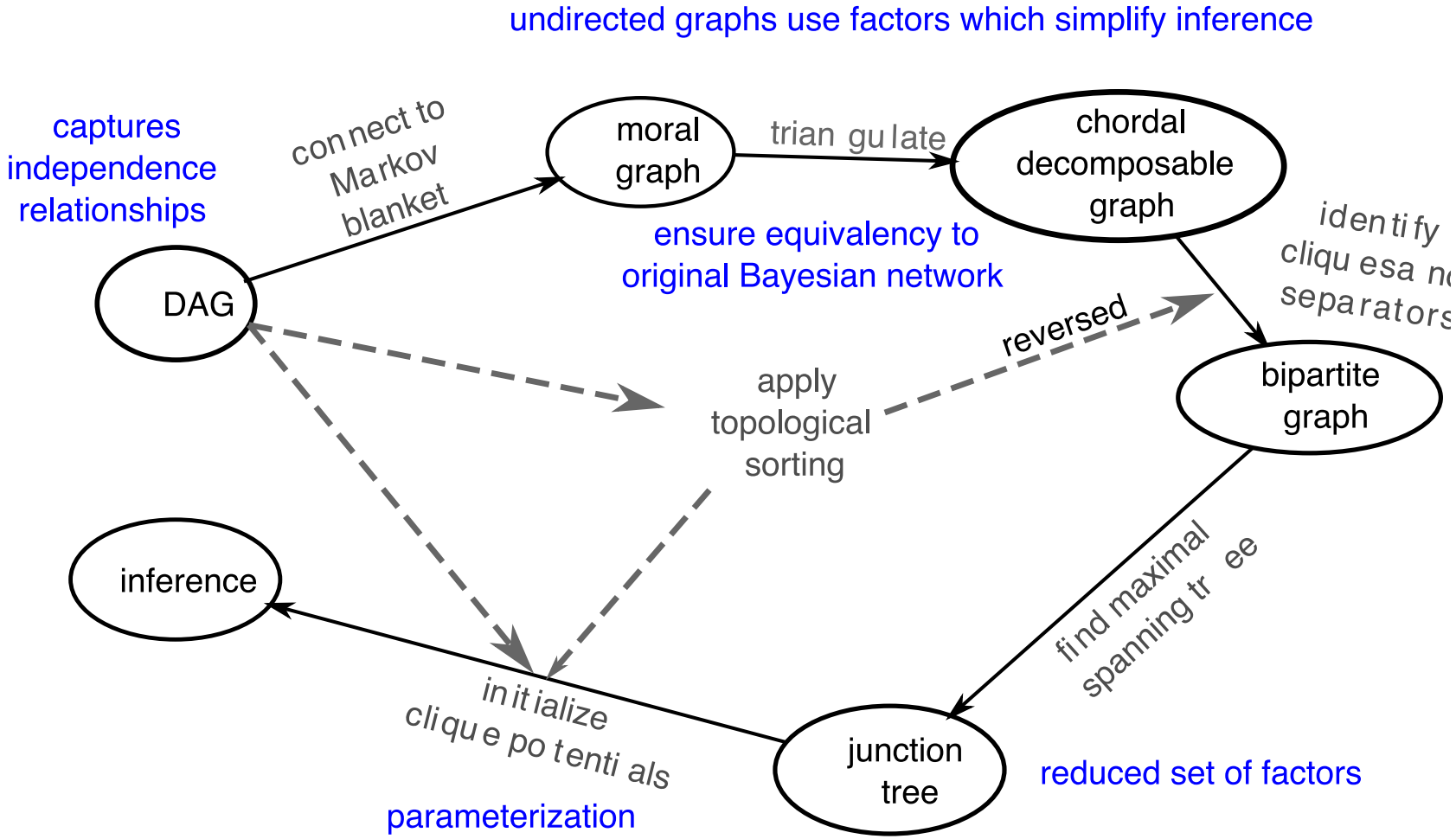
- Last time:
  - What is a graphical model
  - Performing inference using the full joint distribution
  - Product rule, Bayes' Theorem
  - exploiting conditional independence
  - Bayesian Networks (topology, cond. prob. tables)

- Last time:
  - What is a graphical model
  - Performing inference using the full joint distribution
  - Product rule, Bayes' Theorem
  - exploiting conditional independence
  - Bayesian Networks (topology, cond. prob. tables)
- Today:
  - the Junction tree algorithm
  - the message passing algorithm (factor analysis anyone?)

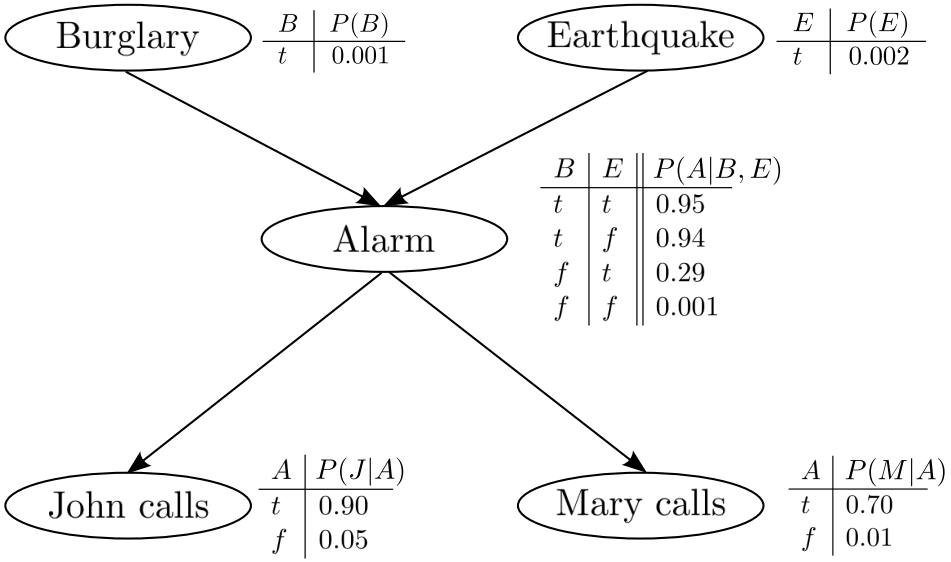
# A road to efficient inference



# A road to efficient inference and WHY



# A road to efficient inference




| $B$ | $P(B)$ |
|---|---|
| $t$ | 0.001 |

| $E$ | $P(E)$ |
|---|---|
| $t$ | 0.002 |

| $B$ | $E$ | $P(A\|B,E)$ |
|---|---|---|
| $t$ | $t$ | 0.95 |
| $t$ | $f$ | 0.94 |
| $f$ | $t$ | 0.29 |
| $f$ | $f$ | 0.001 |

| $A$ | $P(J\|A)$ |
|---|---|
| $t$ | 0.90 |
| $f$ | 0.05 |

| $A$ | $P(M\|A)$ |
|---|---|
| $t$ | 0.70 |
| $f$ | 0.01 |

Figure: DAG of the Californian example

Considering statistical dependencies, only 10 instead of 31 entries have to be stored.

$$P(J,M,A,B,E) \overset{\text{product rule}}{=} P(J|M,A,B,E)\,P(M|A,B,E)\,P(A|B,E)\,P(B|E)\,P(E)$$

$$\overset{\text{cond. indep.}}{=} P(J|A)\,P(M|A)\,P(A|B,E)\,P(B)\,P(E)$$

## Efficient Inference in Bayesian Networks

Bayesian Networks are a factorization of the full joint distribution[1]:

$$P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i | parents(X_i))$$

- We only need to condition on the direct parents of any node $X_i$

[1] See Ch. 14 from (Russell and Norvig, 2016) for more details.

## Efficient Inference in Bayesian Networks

Bayesian Networks are a factorization of the full joint distribution[1]:

$$P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i | parents(X_i))$$

- We only need to condition on the direct parents of any node $X_i$
- The factorization enables us to construct the *directed acyclic graph* (DAG).
- We can also use the DAG to formulate the factorization.

$$DAG_{\text{Bayes.Net}} \quad \widehat{=} \quad \prod_{i=1}^{n} P(X_i | parents(X_i))$$

[1] See Ch. 14 from (Russell and Norvig, 2016) for more details.

## The Junction tree algorithm

We need a representation that scales well for large DAGs to perform marginalization ("summing out") and conditioning more efficiently.
The Junction tree algorithm converts into a tree structure made up of *cliques* and *separators*.

Q: *What is a difference between a DAG and a tree?*

## The Junction tree algorithm

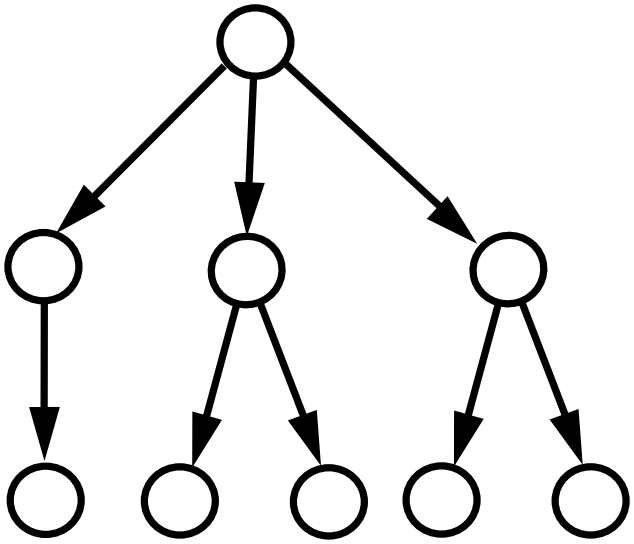
We need a representation that scales well for large DAGs to perform marginalization ("summing out") and conditioning more efficiently.
The Junction tree algorithm converts into a tree structure made up of *cliques* and *separators*.

Q: *What is a difference between a DAG and a tree?*

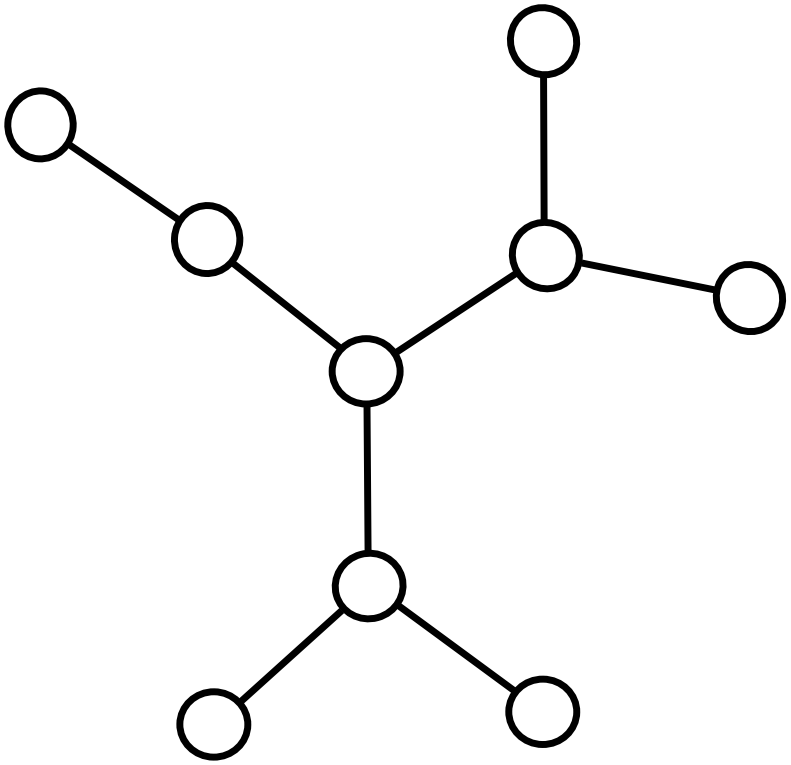
- A tree structure guarantees that the path between any two nodes is unique.

## Trees

- **tree**: graph where each existing path between nodes is *unique*

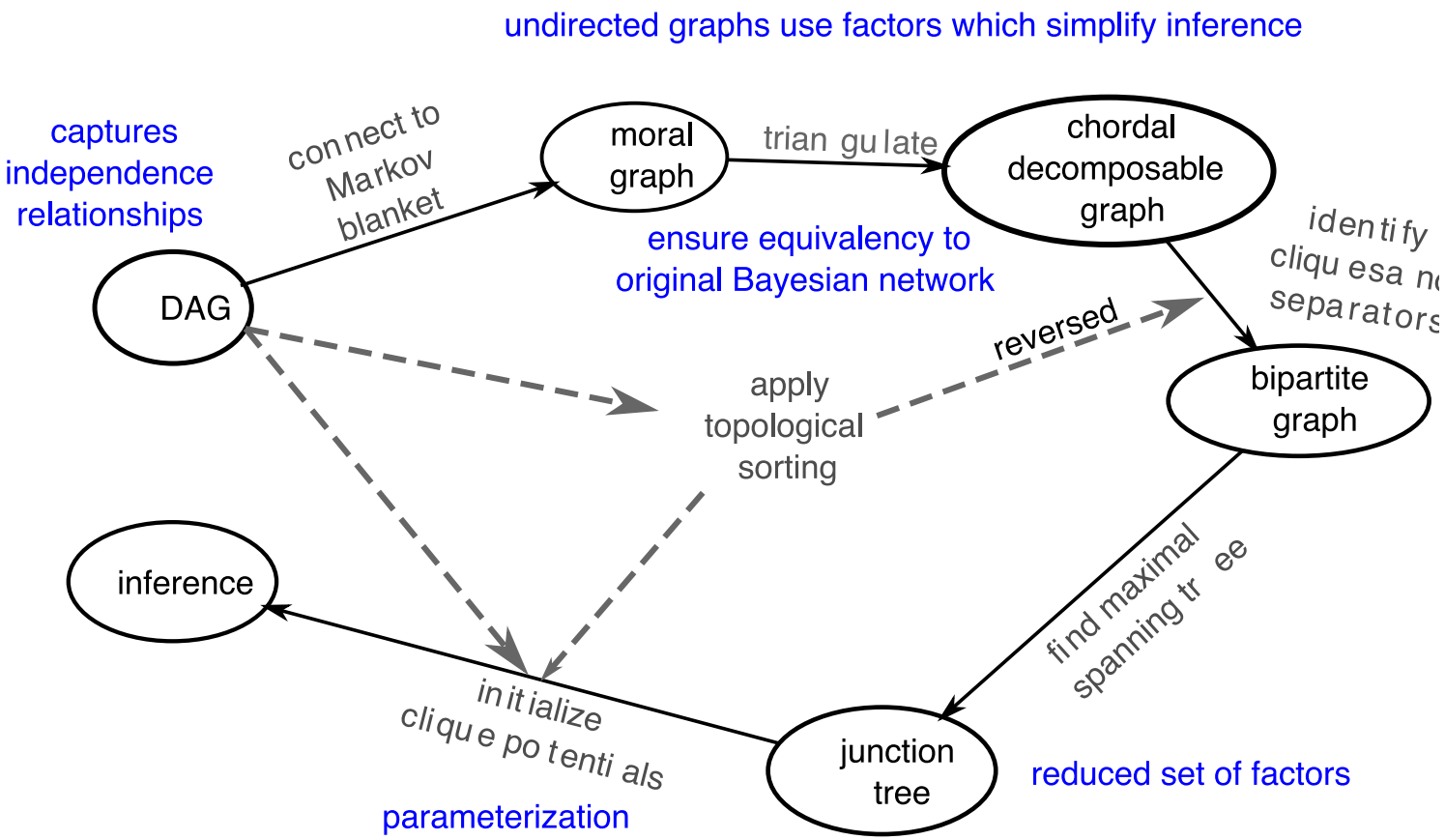


directed tree



undirected tree

# Decomposable Undirected Graphs



## Undirected Graph

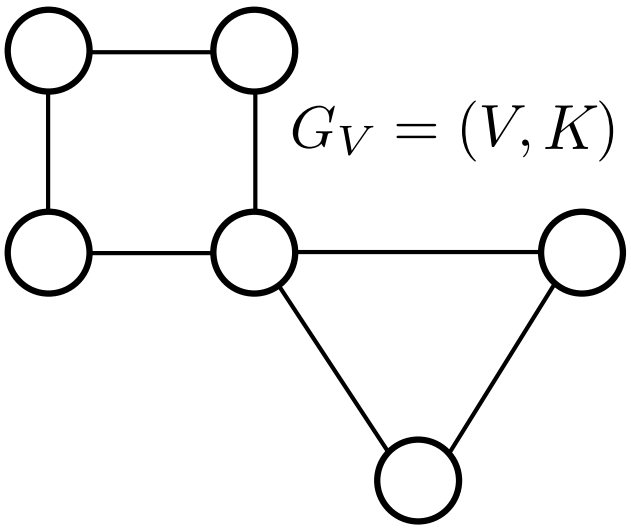
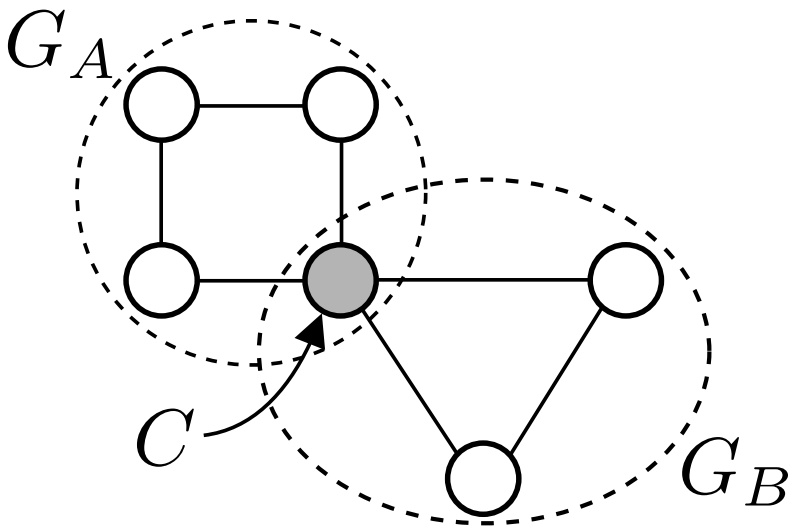



Figure: undirected graph $G_V$ with vertices $V$ and edges $K$

## Separators

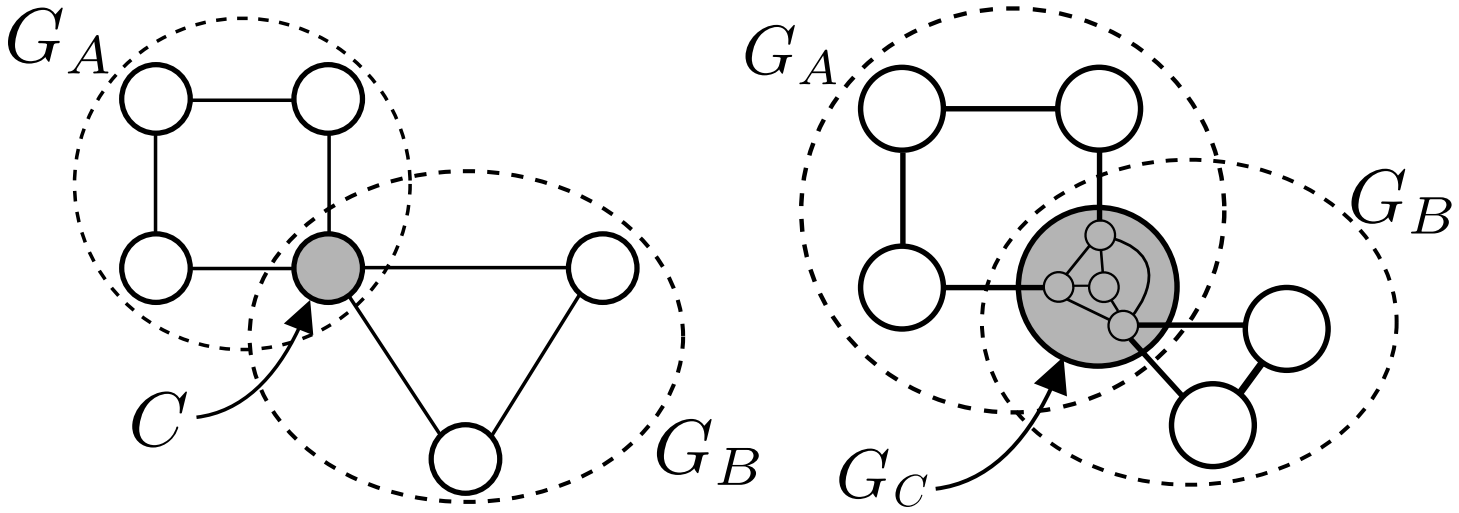



node C separates subsets A and B

### Separator

A set $C$ of nodes **separates** two undirected subgraphs $G_A$ and $G_B$ if *every* path from node set $A$ to node set $B$ has to pass through an element of $C$.

## Separators




(a) node C (b) set of nodes C

Figure: subgraphs A and B separated by C

### Separator

A set $C$ of nodes **separates** two undirected subgraphs $G_A$ and $G_B$ if *every* path from node set $A$ to node set $B$ has to pass through an element of $C$.

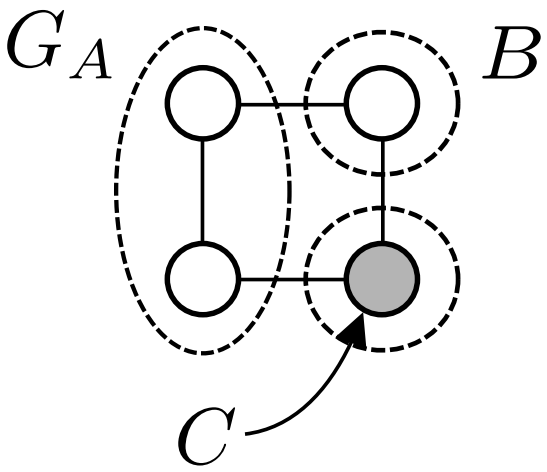
# Decomposable graphs

- $A, B, C$ are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - $A, B, C$ are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - $C$ separates $G_A$ and $G_B$, and
  - subgraph $G_C$ is complete.
- $G$ is **decomposable** if it is complete, or a proper decomposition $A, B, C$ exists, <u>where</u> $G_{A \cup C}$ and $G_{B \cup C}$ are decomposable.

# Decomposable graphs

- $A, B, C$ are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - $A, B, C$ are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - $C$ separates $G_A$ and $G_B$, and
  - subgraph $G_C$ is complete.
- $G$ is **decomposable** if it is complete, or a proper decomposition $A, B, C$ exists, where $G_{A \cup C}$ and $G_{B \cup C}$ are decomposable.

Q: *Are the following (i) a proper decomposition, (ii) decomposable graphs?*



## Decomposable graphs

- $A, B, C$ are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - $A, B, C$ are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - $C$ separates $G_A$ and $G_B$, and
  - subgraph $G_C$ is complete.
- $G$ is **decomposable** if it is complete, or a proper decomposition $A, B, C$ exists, where $G_{A \cup C}$ and $G_{B \cup C}$ are decomposable.

Q: *Are the following (i) a proper decomposition, (ii) decomposable graphs?*

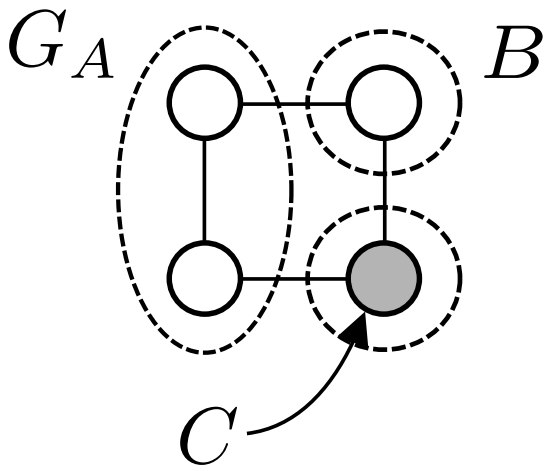
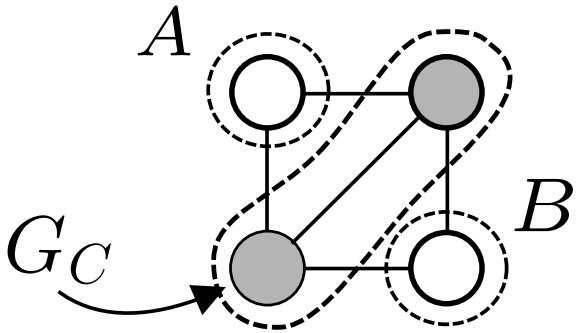



Figure: not a proper decomposition and not decomposable

# Decomposable graphs

- $A, B, C$ are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - $A, B, C$ are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - $C$ separates $G_A$ and $G_B$, and
  - subgraph $G_C$ is complete.
- $G$ is **decomposable** if it is complete, or a proper decomposition $A, B, C$ exists, where $G_{A \cup C}$ and $G_{B \cup C}$ are decomposable.

Q: *Are the following (i) a proper decomposition, (ii) decomposable graphs?*



## Decomposable graphs

- $A, B, C$ are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - $A, B, C$ are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - $C$ separates $G_A$ and $G_B$, and
  - subgraph $G_C$ is complete.
- $G$ is **decomposable** if it is complete, or a proper decomposition $A, B, C$ exists, where $G_{A \cup C}$ and $G_{B \cup C}$ are decomposable.

Q: *Are the following (i) a proper decomposition, (ii) decomposable graphs?*

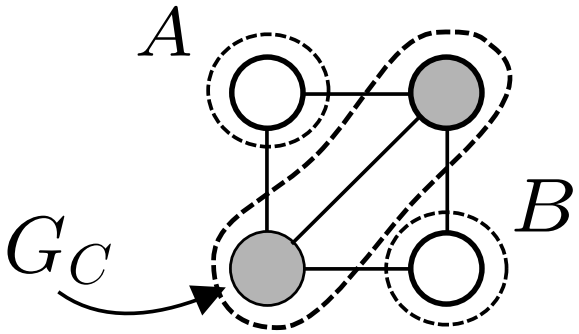
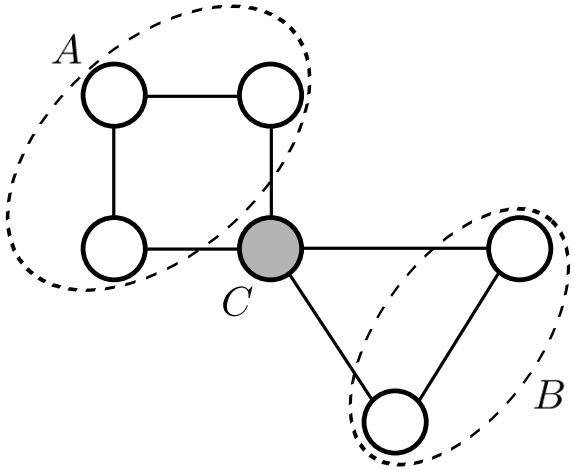



Figure: a proper decomposition and decomposable

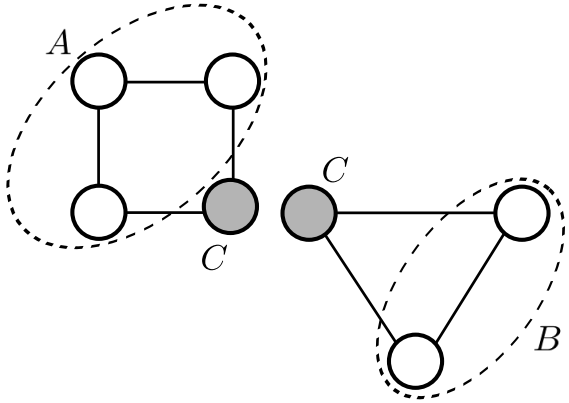
# Decomposable graphs

- $A, B, C$ are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - $A, B, C$ are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - $C$ separates $G_A$ and $G_B$, and
  - subgraph $G_C$ is complete.
- $G$ is **decomposable** if it is complete, or a proper decomposition $A, B, C$ exists, <u>where</u> $G_{A \cup C}$ and $G_{B \cup C}$ are decomposable.

Q: *Are the following (i) a proper decomposition, (ii) decomposable graphs?*




(a) example decomposition

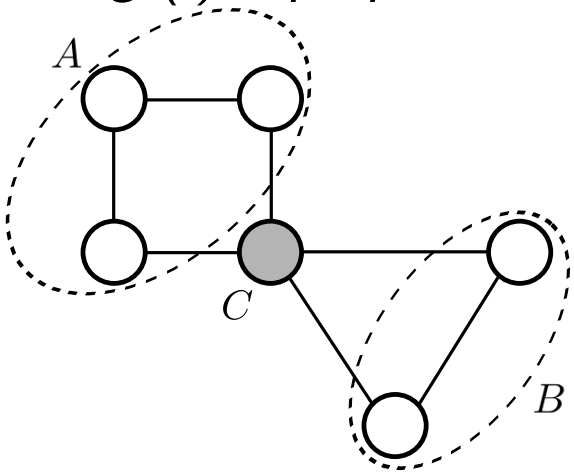



(b) <u>Hint</u>

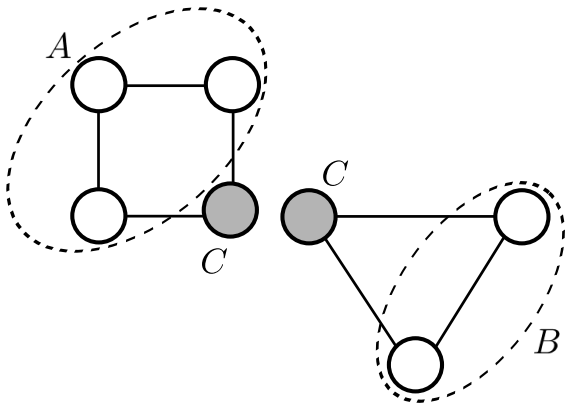
## Decomposable graphs

- $A, B, C$ are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - $A, B, C$ are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - $C$ separates $G_A$ and $G_B$, and
  - subgraph $G_C$ is complete.
- $G$ is **decomposable** if it is complete, or a proper decomposition $A, B, C$ exists, where $G_{A \cup C}$ and $G_{B \cup C}$ are decomposable.

Q: *Are the following (i) a proper decomposition, (ii) decomposable graphs?*




(a) example decomposition



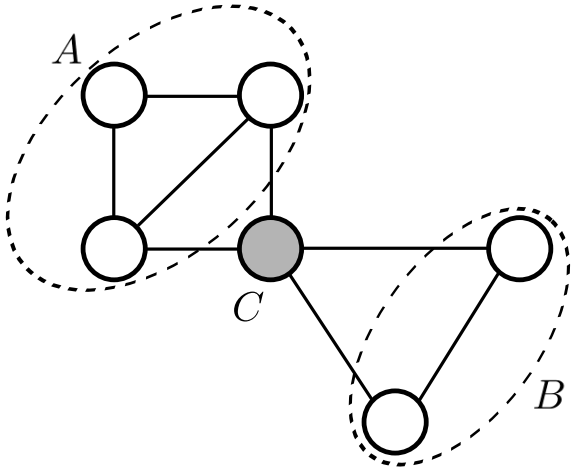

(b) Hint : because $G_{A \cup C}$ is not decomposable

Figure: a proper decomposition but not decomposable

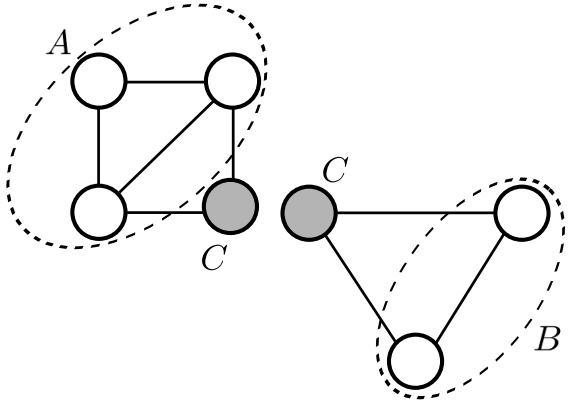
# Decomposable graphs

- $A, B, C$ are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - $A, B, C$ are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - $C$ separates $G_A$ and $G_B$, and
  - subgraph $G_C$ is complete.
- $G$ is **decomposable** if it is complete, or a proper decomposition $A, B, C$ exists, where $G_{A \cup C}$ and $G_{B \cup C}$ are decomposable.

Q: *Are the following (i) a proper decomposition, (ii) decomposable graphs?*




(a) example decomposition

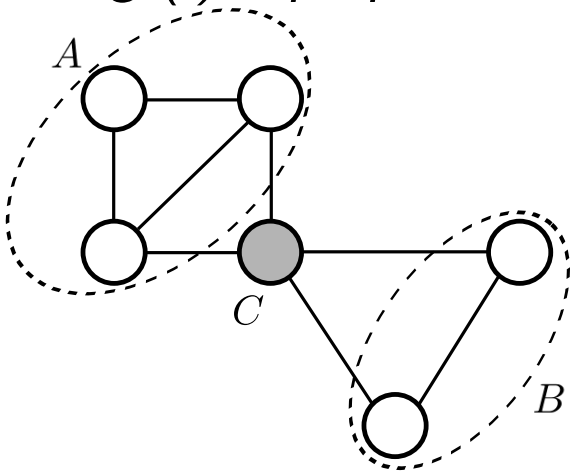



(b) Hint

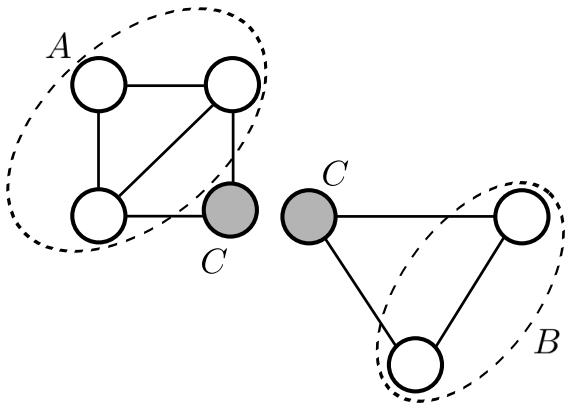
# Decomposable graphs

- $A, B, C$ are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - $A, B, C$ are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - $C$ separates $G_A$ and $G_B$, and
  - subgraph $G_C$ is complete.
- $G$ is **decomposable** if it is complete, or a proper decomposition $A, B, C$ exists, where $G_{A \cup C}$ and $G_{B \cup C}$ are decomposable.

Q: *Are the following (i) a proper decomposition, (ii) decomposable graphs?*




(a) example decomposition



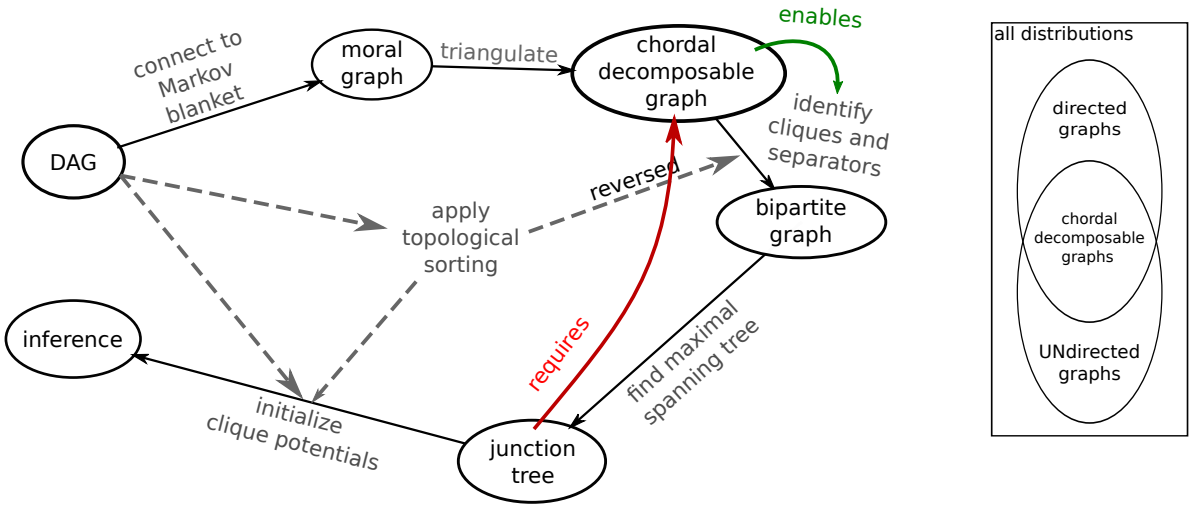

(b) Hint : because $G_{A \cup C}$ is decomposable and $G_{B \cup C}$ is complete

## Decomposable graphs

Q: *Why are we interested in decomposable graphs?*

# Decomposable graphs

Q: *Why are we interested in decomposable graphs?*




- The short answer:
  - a it is a requirement for the construction of junction trees
  - b decomposable graphs allow us to form a *bipartite graph* of cliques and separators.

# Efficient Inference in Bayesian Networks

## Cliques and separators

- **cliques** are maximally complete subgraphs
- decomposable graphs can be decomposed into cliques and separators
- cliques and separators form a *bipartite graph* (cf. Fig. 7)

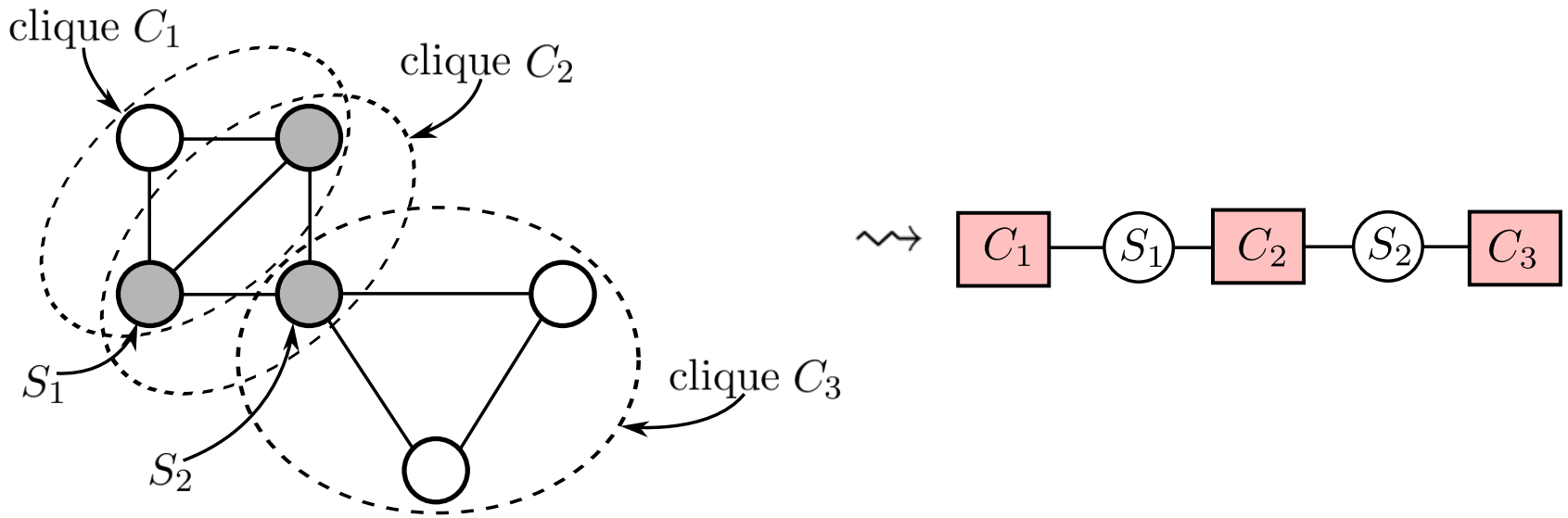
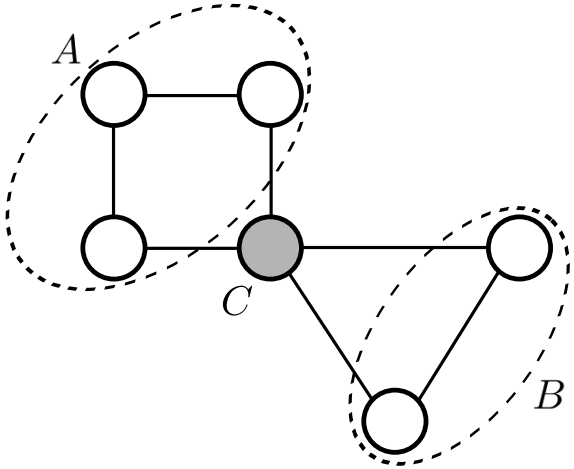
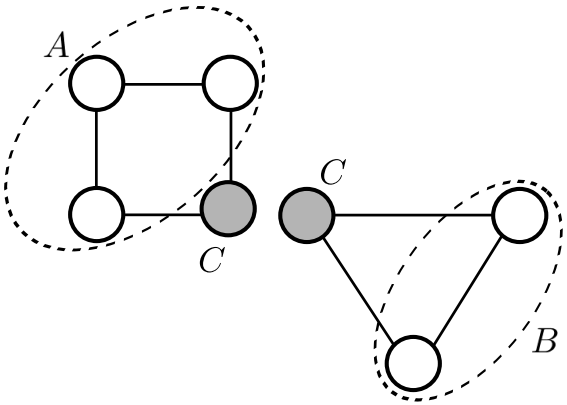



Figure: a decomposable graph and corresponding bipartite graph

# Decomposable graphs




(a) example decomposition




(b) because $G_{A \cup C}$ is not decomposable

Figure: a proper decomposition but not decomposable

Q: *What if we don't have a decomposable graph ?*

## Decomposable graphs

Q: *What if we don't have a decomposable graph ?*

- We apply triangulation. We add a *chord*, i.e. "shortcut" edges to eliminate any "circles" (i.e. cycles) of 4+ vertices.

[2] Koller and Friedman, 2009, Ch 9.4.3

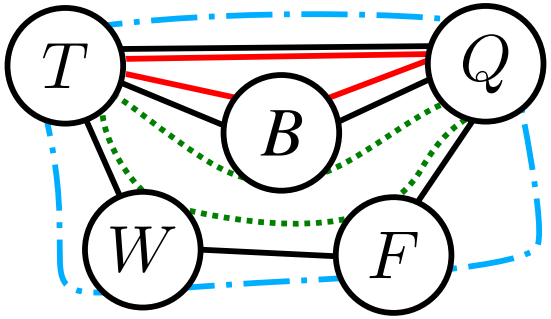
## Decomposable graphs

Q: *What if we don't have a decomposable graph ?*

- We apply triangulation. We add a *chord*, i.e. "shortcut" edges to eliminate any "circles" (i.e. cycles) of 4+ vertices.
A heuristic to keep the sizes of cliques small.
Smaller cliques $\rightsquigarrow$ "modular" bipartite graph $\rightsquigarrow$ faster computation during inference.[2]

[2] Koller and Friedman, 2009, Ch 9.4.3

## Cycles and chords




- *Cycle*: A closed path with **no** repeated vertices except $V_{\text{start}} = V_{\text{end}}$.
- *Chords*: Given a cycle, an edge is a chord if it connects two nodes that are non-adjacent on the cycle.
- *Chordless* cycle: **All** two **non**-adjacent vertices on the cycle are **not** joined by any edge.
- *Triangulated graph* ($\widehat{=}$ *Chordal graph*): A graph **without** chordless cycles of 4+ vertices.

(too many double negatives!!!)

### Triangulation

Construct a chordal graph by adding *chords* ("shortcut" edges) to all cycles of length 4+.

# Triangulation: Examples

## Triangulation

Construct a chordal graph by adding *chords* ("shortcut" edges) all cycles of length 4+.

Consider the following graph with vertices $A, B, C, E$ and add chords where necessary: **(see blackboard)**

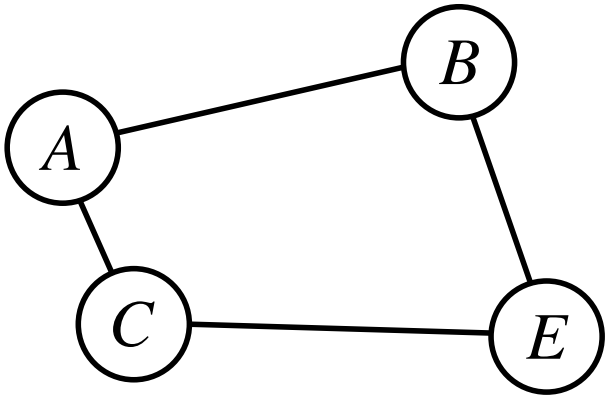


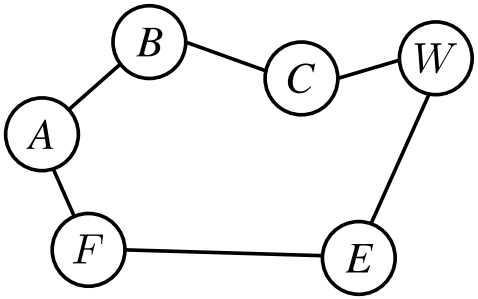

Figure: a non-decomposable graph

# Triangulation: Examples

## Triangulation

Construct a chordal graph by adding *chords* ("shortcut" edges) all cycles of length 4+.

Consider the following graph with vertices $A, B, C, W, E, F$ and add chords where necessary:
**(see blackboard)**



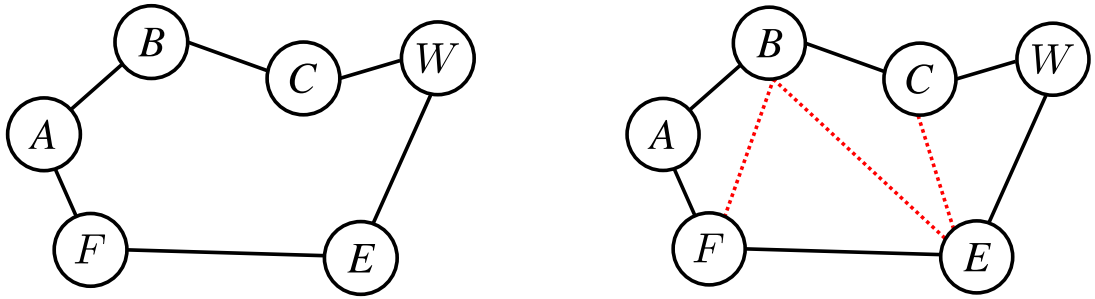

(a) example graph to triangulate

# Triangulation: Examples

## Triangulation

Construct a chordal graph by adding *chords* ("shortcut" edges) all cycles of length 4+.

Consider the following graph with vertices $A, B, C, W, E, F$ and add chords where necessary:
**(see blackboard)**




(a) example graph to triangulate

(b) One possible and non-unique triangulation

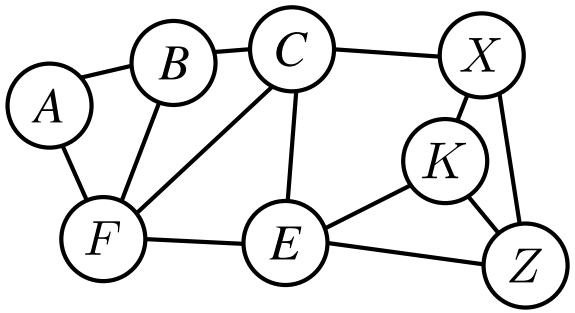
Figure: Triangulation example

# Triangulation: Examples

## Triangulation

Construct a chordal graph by adding *chords* ("shortcut" edges) all cycles of length 4+.

Q: *Is this a chordal graph? i.e. does it contain chordless cycles with 4+ vertices?*



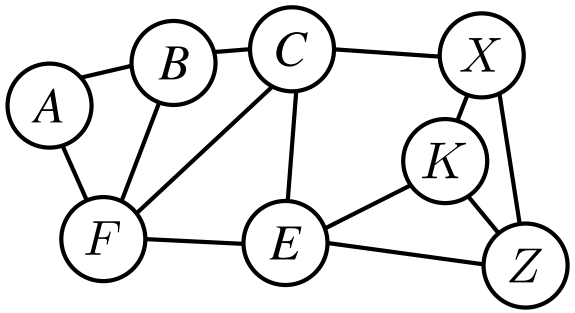

(a) example graph

# Triangulation: Examples

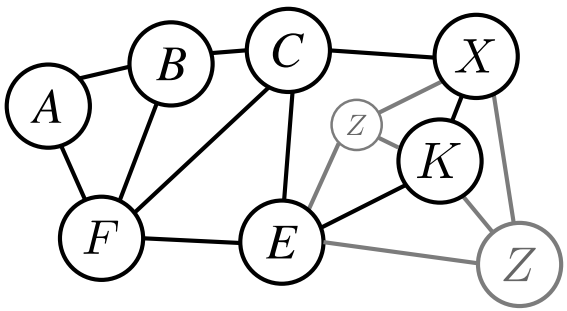
## Triangulation

Construct a chordal graph by adding *chords* ("shortcut" edges) all cycles of length 4+.

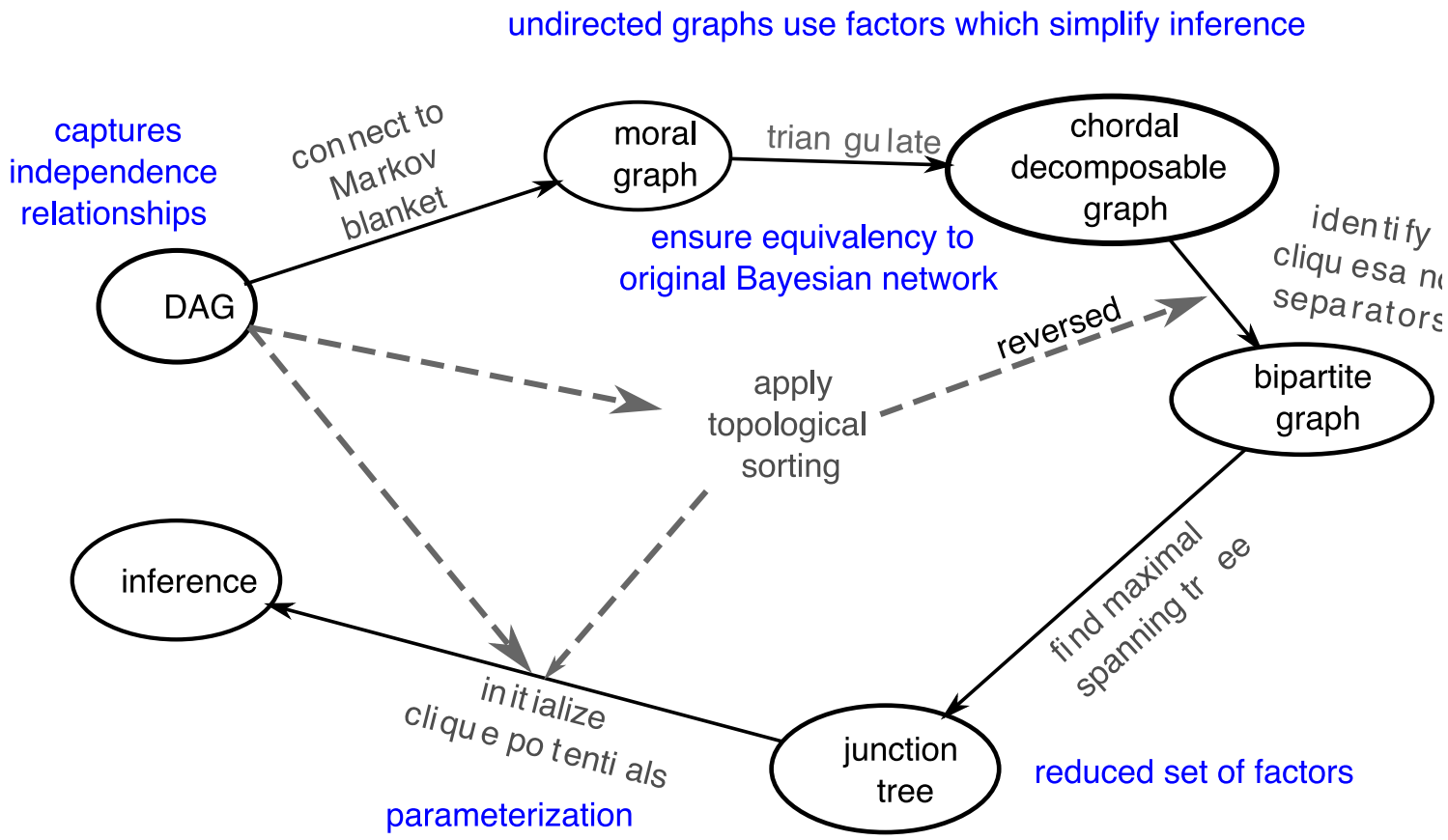
Q: *Is this a chordal graph? i.e. does it contain chordless cycles with 4+ vertices?*




(a) example graph



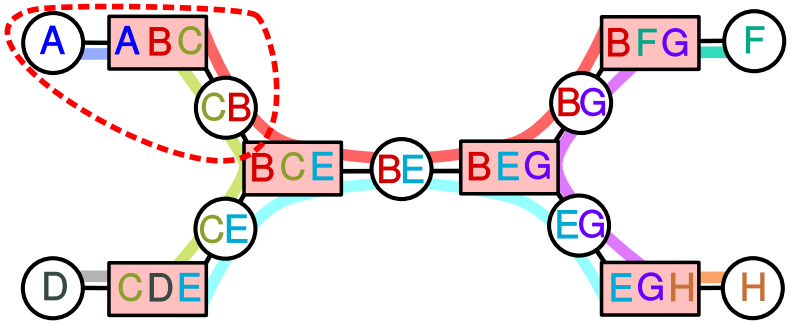
# Construction of the Junction Tree



# The running intersection property

**The running intersection property**

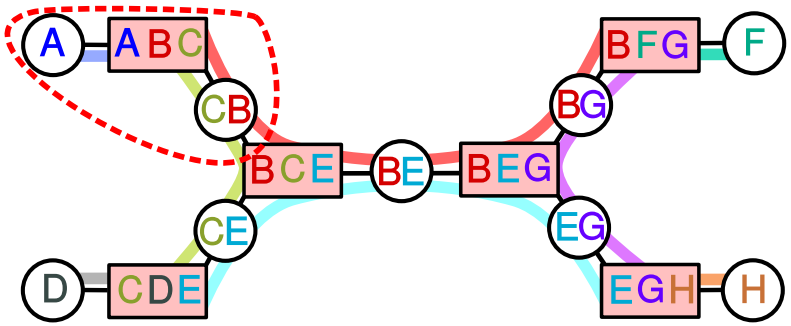
All nodes on the path between two cliques, which both contain variable $X$, also contain $X$.



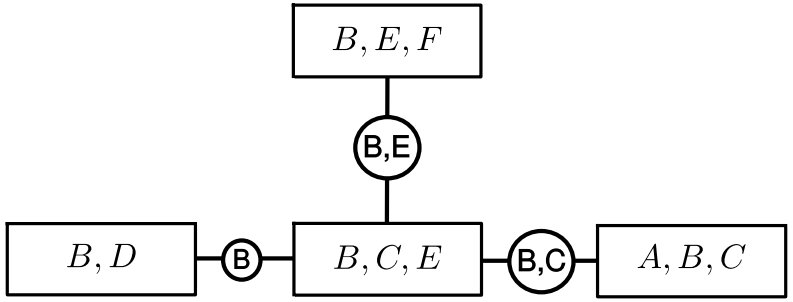
# The running intersection property

## The running intersection property

All nodes on the path between two cliques, which both contain variable $X$, also contain $X$.



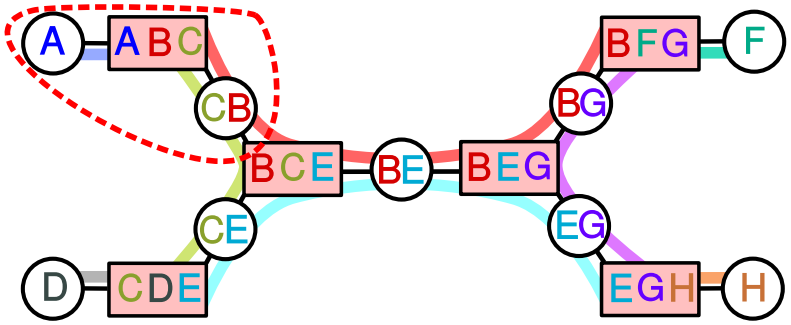

Q: *Is the following a valid junction tree?*



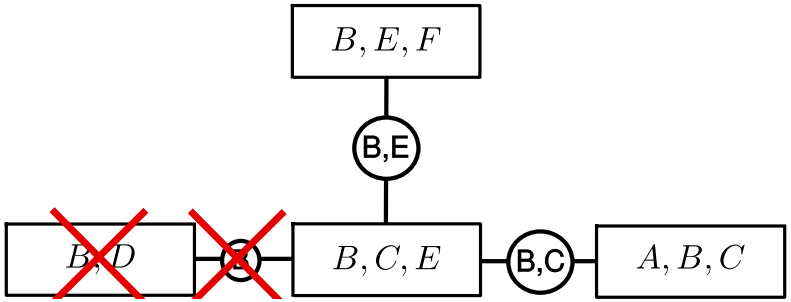
# The running intersection property

## The running intersection property

All nodes on the path between two cliques, which both contain variable $X$, also contain $X$.



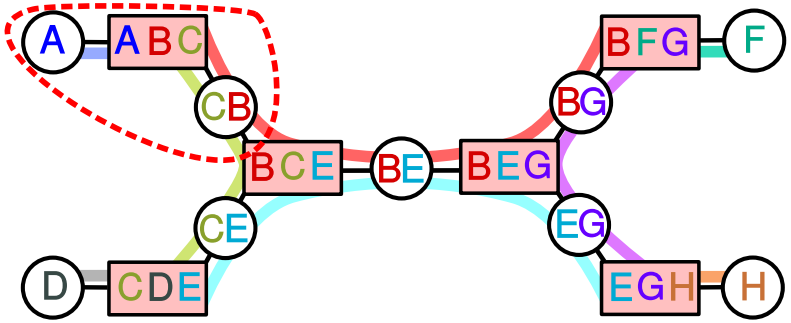

Q: *Which of the following remains a valid junction tree? (irrespective of being useful for exact inference)*



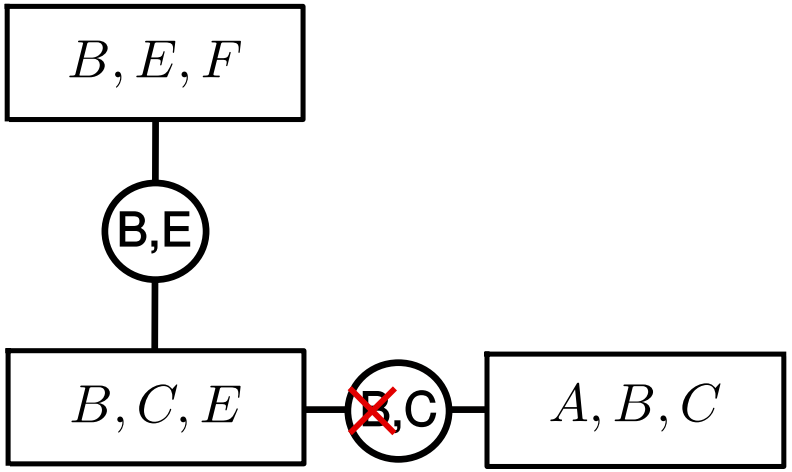
# The running intersection property

## The running intersection property

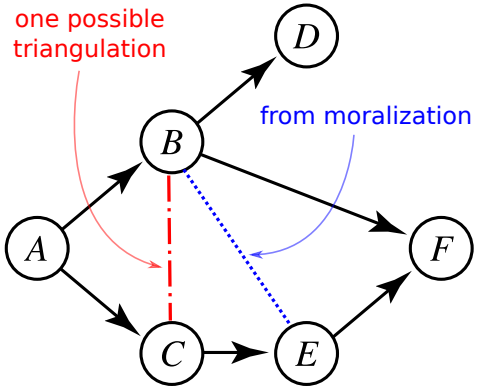
All nodes on the path between two cliques, which both contain variable $X$, also contain $X$.




Q: *Which of the following remains a valid junction tree? (irrespective of being useful for exact inference)*



# Identify cliques and separators to form a bipartite graph



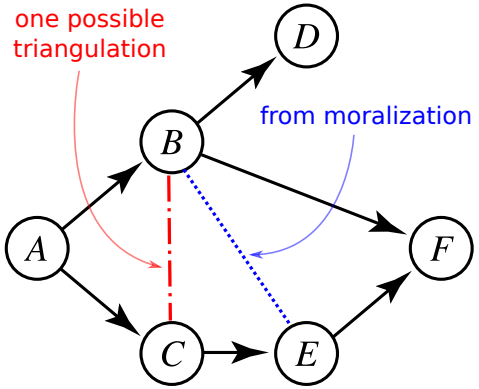

- cliques are maximally complete subgraphs (sub-cliques are automatically absorbed)
- cliques can be found by elimination

**Caution: only eliminate a node if it is not part of another clique**

- requires variable ordering by (**reverse**) topological sorting: F, D, E, C, B, A

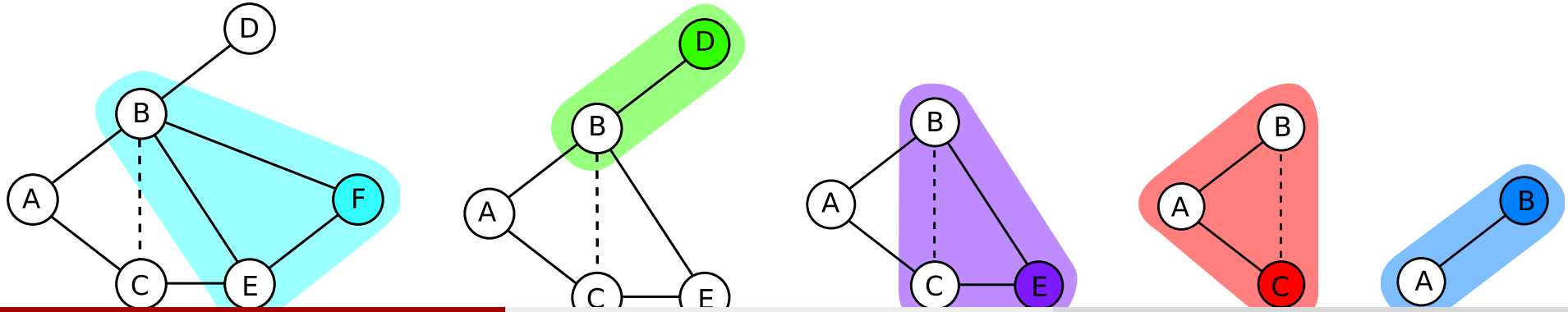
# Identify cliques and separators to form a bipartite graph

- cliques are maximally complete subgraphs (sub-cliques are automatically absorbed)
- cliques can be found by elimination



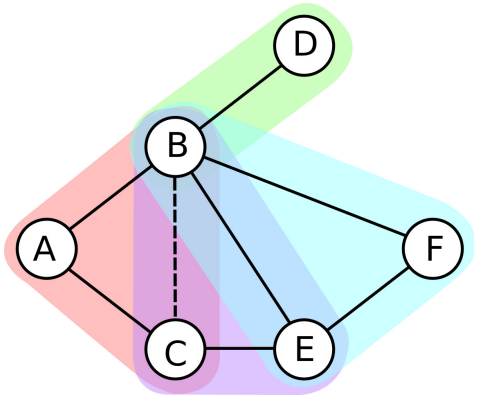

**Caution: only eliminate a node if it is not part of another clique**

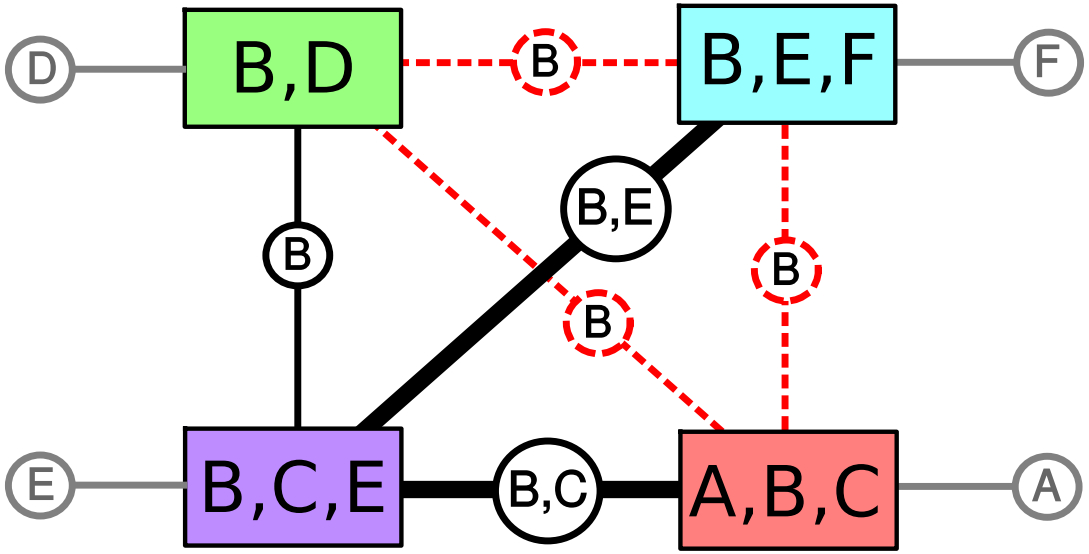
- requires variable ordering by (**reverse**) topological sorting: F, D, E, C, B, A



# From bipartite graph to junction tree

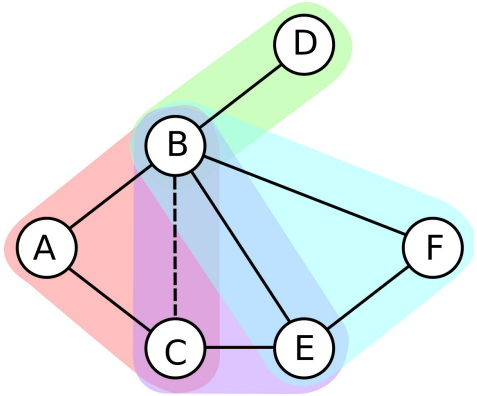
- construct the bipartite graph:
  - edges weighted by separator size
    $\rightarrow$ number of nodes within separator
  - find a *maximal spanning tree*

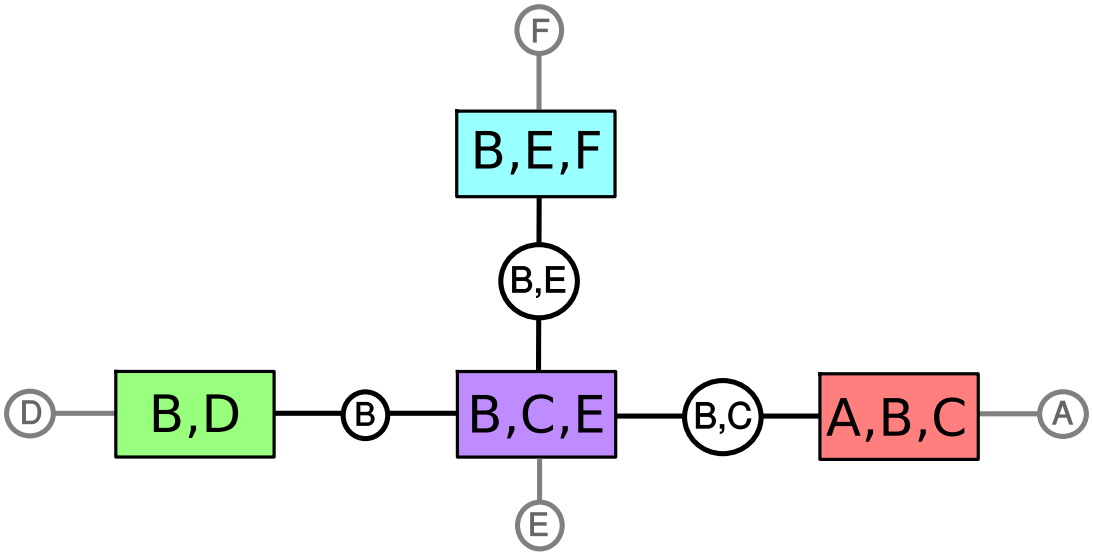


# From bipartite graph to junction tree

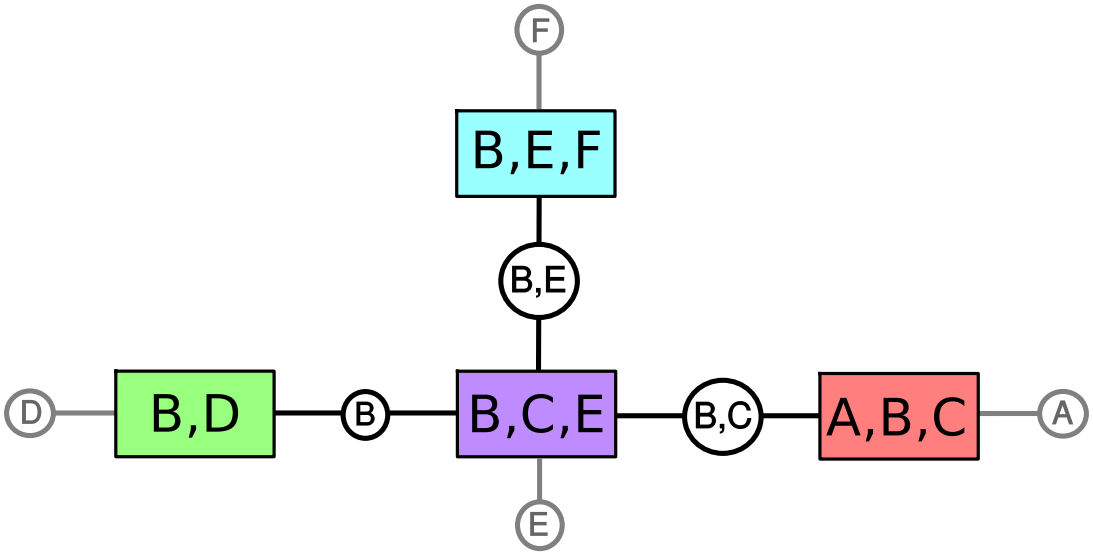
- construct the bipartite graph:
- junction tree for inference
  - maximal spanning tree of the clique graph
  - data structure for belief propagation
  - not always unique



## Inference in a junction tree

1. clique potentials (non-negative function) for each clique (get definitions from DAG)
2. order initialization according to topological sorting (parent $\rightarrow$ child)
3. add clique potential(s) for observed evidence(s)
4. perform message passing
5. extract marginals from messages
6. perform normalization



# Message Passing

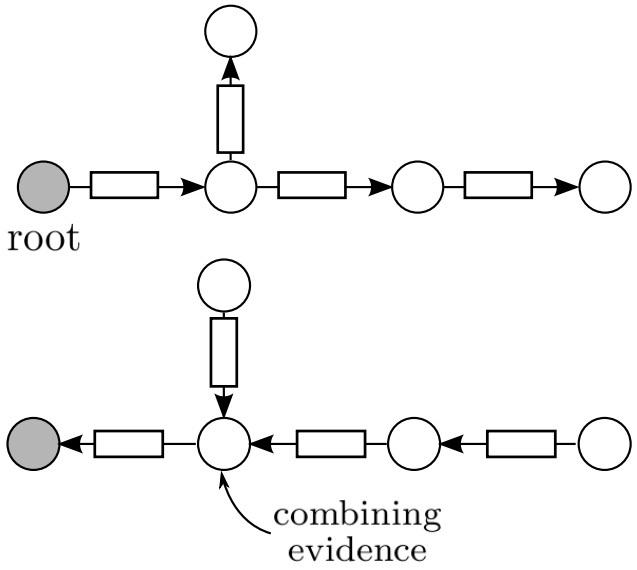
a compact look-up-table

## Message Passing: Overview

- essentially storage of local computations for faster look-up
- will be used to calculate conditional probabilities in a junction tree by only using local "simpler" computations.
- scales well with no. of variables.

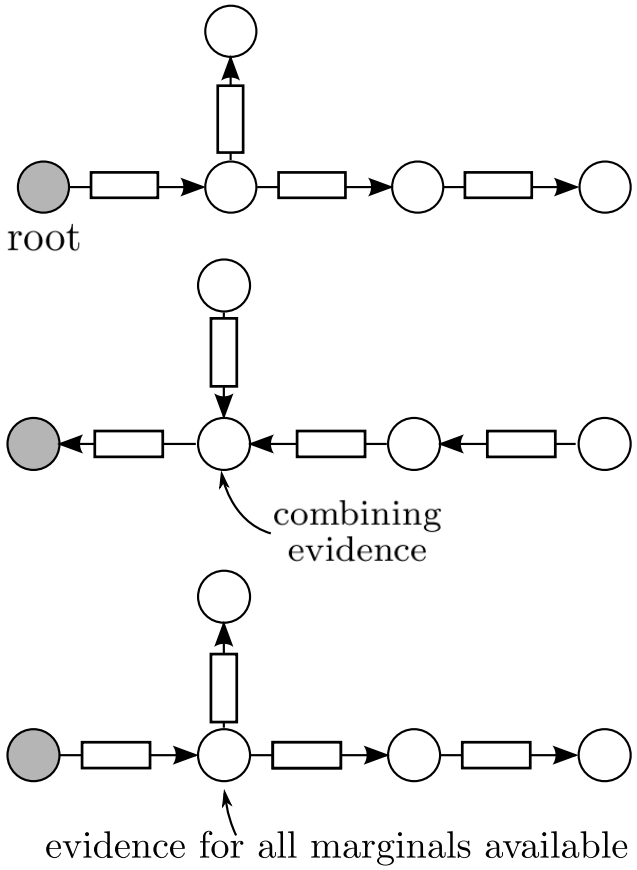
# Message Passing: Procedure

- first pass from root to leaves: "request" selecting the root (for tree structures any node can be the root, so not sensitive to choice of root node)
- second pass from leaves to root: "collect" collect messages from other neighbors



# Message Passing: Procedure

- first pass from root to leaves: "request"
  selecting the root (for tree structures any node can be the root, so not sensitive to choice of root node)
- second pass from leaves to root: "collect"
  collect messages from other neighbors
- a third pass can calculate all other marginals: "distribute"
  third pass from root to leaves to compute the remaining messages.



## The sum-product algorithm

Defining the messages between the different types of nodes in a bipartite graph (e.g. separators and cliques)[3]

$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

[3] for details, see Ch. 8.4.4, p. 402 in (Bishop, 2006)

## The sum-product algorithm

Defining the messages between the different types of nodes in a bipartite graph (e.g. separators and cliques)[3]

$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

[3] for details, see Ch. 8.4.4, p. 402 in (Bishop, 2006)

## The sum-product algorithm

Defining the messages between the different types of nodes in a bipartite graph (e.g. separators and cliques)[3]

$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k)$$

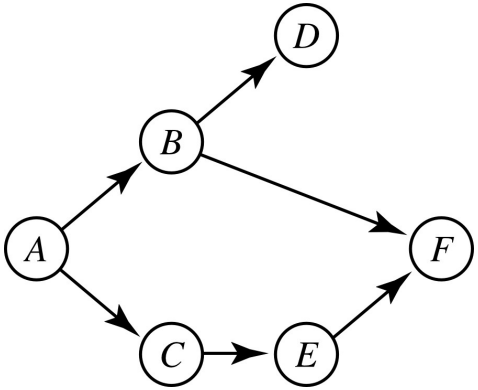
[3] for details, see Ch. 8.4.4, p. 402 in (Bishop, 2006)

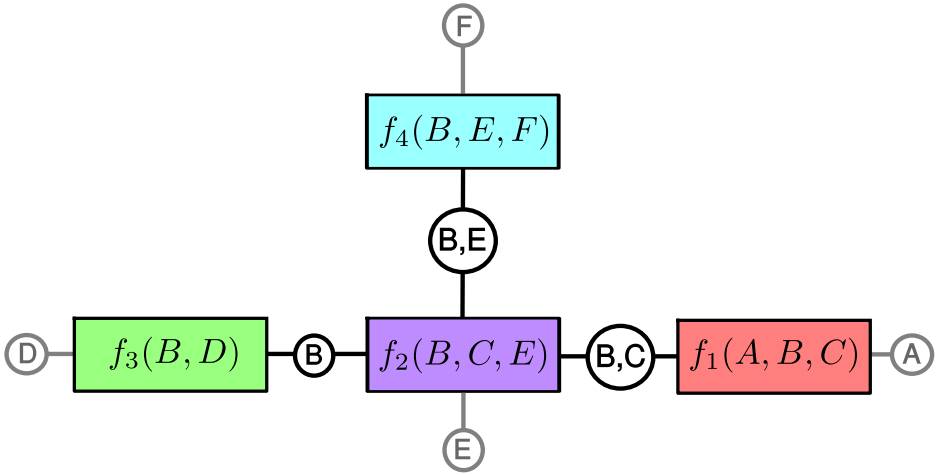
## Inference

1. initialization of the clique potentials $f_k(\mathcal{X}_k)$
   - in the order established by topological sorting (e.g. $A \to B \to C \to E \to D \to F$)

| $f_1(A,B,C)$ | $f_2(B,C,E)$ | $f_3(B,D)$ | $f_4(B,E,F)$ |
|---|---|---|---|
| $P(C\|A)\,P(B\|A)\,P(A)$ | $P(E\|C)$ | $P(D\|B)$ | $P(F\|B,E)$ |

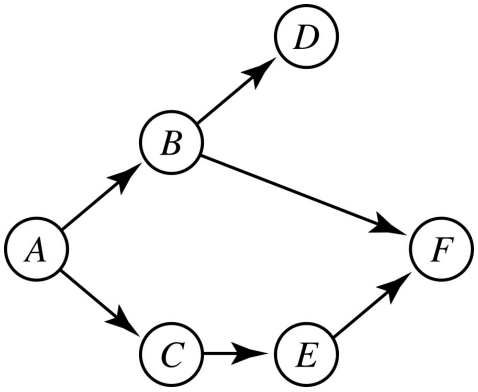
$$P(A,B,C,D,E,F) \;=\; P(A)\,P(B|A)\,P(C|A)\,P(D|B)\,P(E|C)\,P(F|B,E)$$

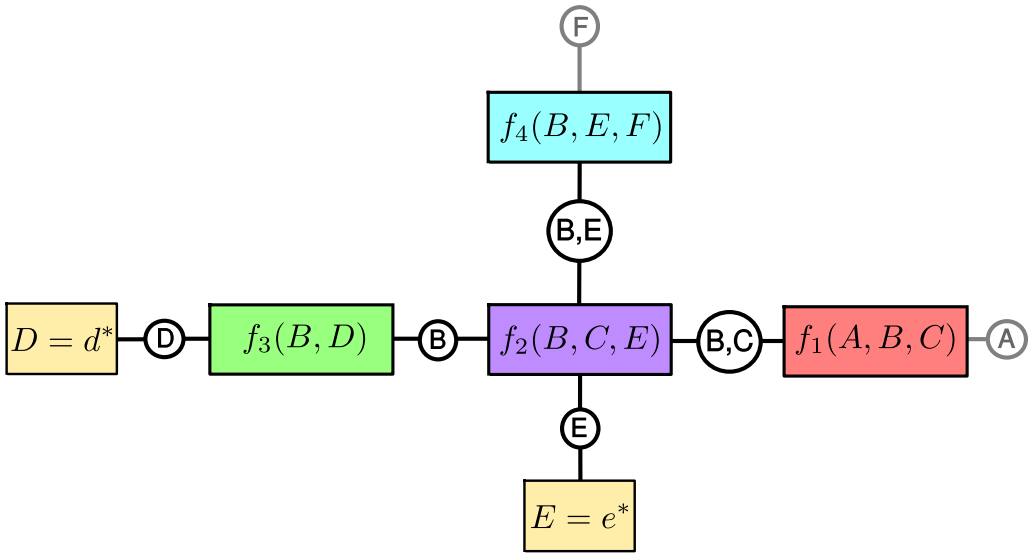


## Inference

1. initialization of the clique potentials $f_k(\mathcal{X}_k)$
2. modification of the clique potentials by the observed evidence
   - for each observation $Y = y^*$ find *one* $f_k$ with $Y \in \mathcal{X}_k$
   - add a node $f_k(Y) = \begin{cases} 1, & Y = y^* \\ 0, & Y \neq y^* \end{cases}$
   - example: $\boxed{D = d^*}$ and $\boxed{E = e^*}$

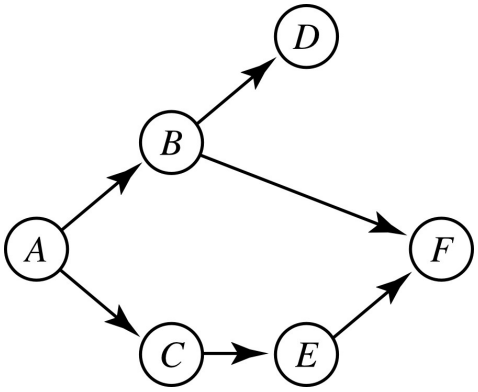


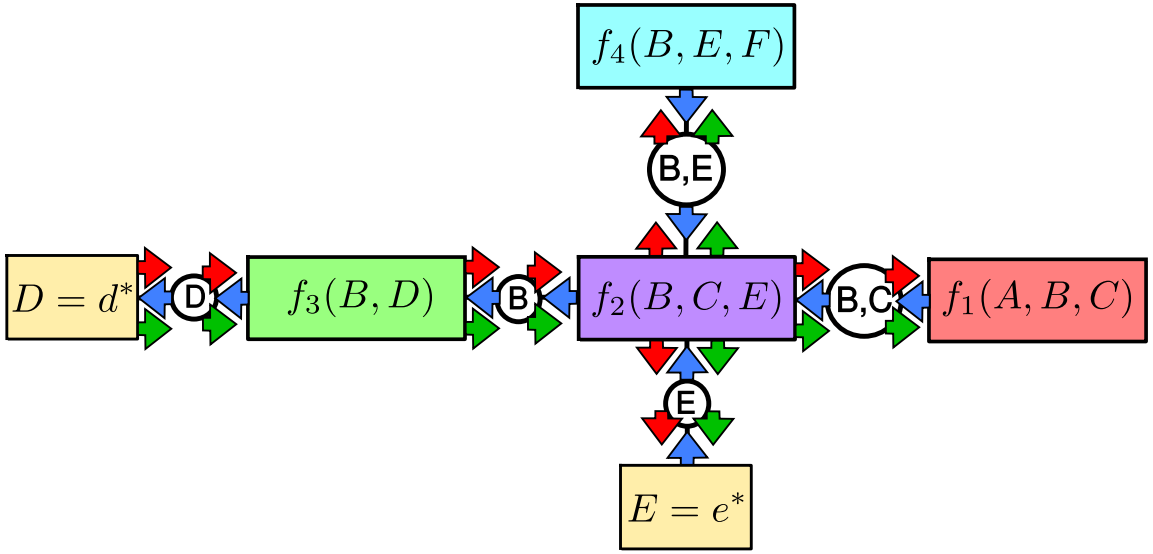



## Inference

1. initialization of the clique potentials $f_k(\mathcal{X}_k)$
2. modification of the clique potentials by the observed evidence
3. message passing
   - begin "request" pass from arbitrary node $N$, e.g. $\boxed{D = d^*}$ or $\boxed{f_1}$ or ...
   - wait for all message of "collect" pass to return
   - send last "distribute" pass from $N$

   $\Rightarrow$ all marginals are computed simultaneously

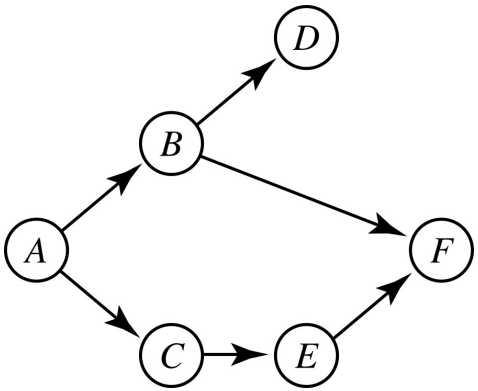


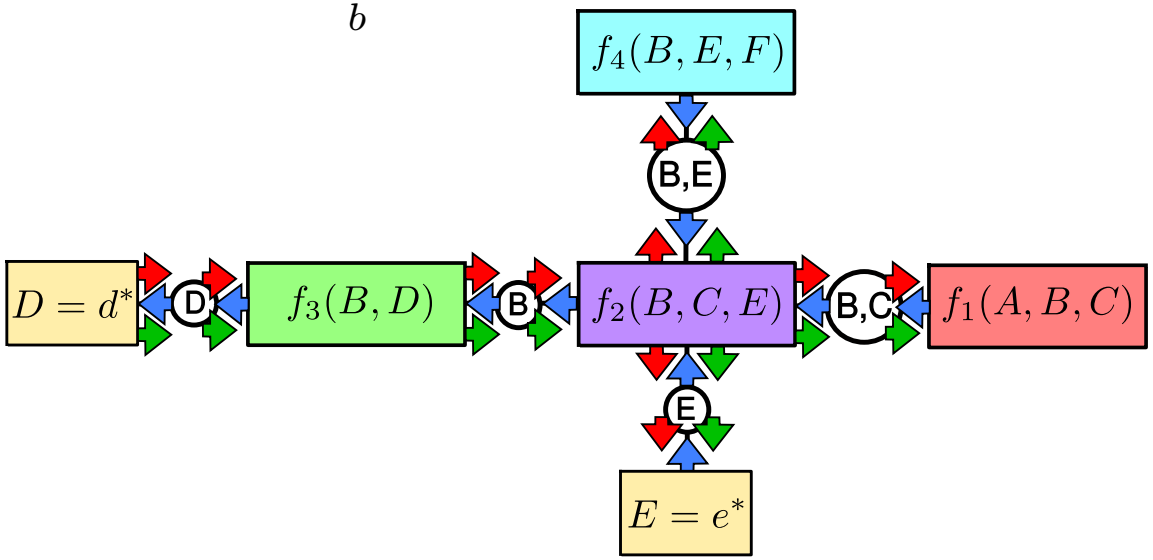



## Inference

1. initialization of the clique potentials $f_k(\mathcal{X}_k)$
2. modification of the clique potentials by the observed evidence
3. message passing
4. calculate marginals from messages

$$P(C, D = d^*, E = e^*) \quad = \quad \sum_b \mu_{f_1 \to BC}(b, C) \cdot \mu_{f_2 \to BC}(b, C)$$

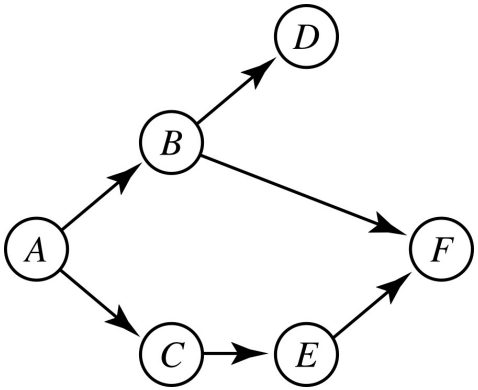


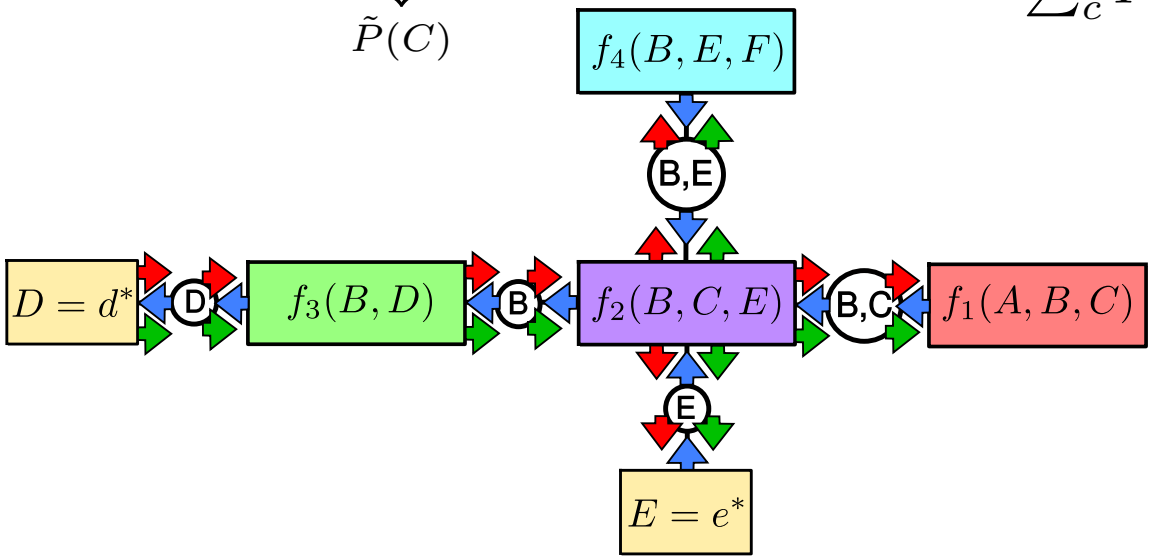



## Inference

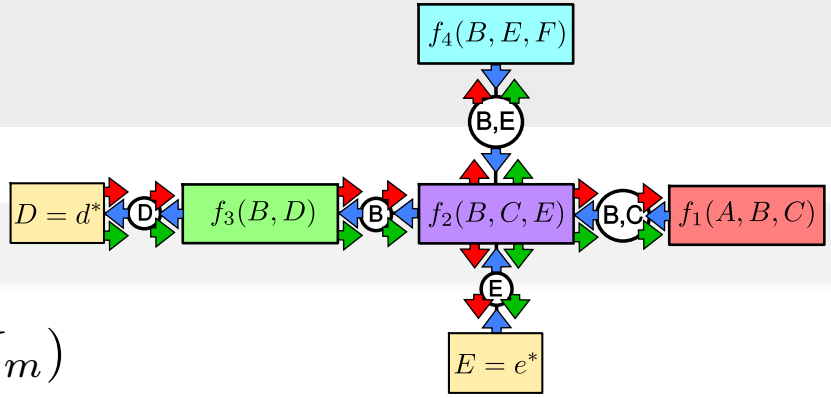
1. initialization of the clique potentials $f_k(\mathcal{X}_k)$
2. modification of the clique potentials by the observed evidence
3. message passing
4. calculate marginals from messages
5. normalization yields conditional marginals

$$P(C\,|D = d^*, E = e^*) = \alpha\, \underbrace{P(C,\, D = d^*, E = e^*)}_{\tilde{P}(C)}, \qquad \alpha = \frac{1}{\sum_c \tilde{P}(c)}$$



# Message passing exercise




## sum-product

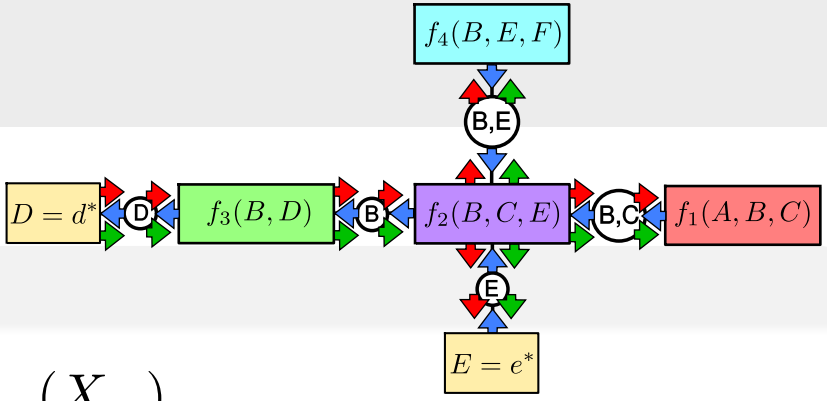
$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k)$$

Define the messages by applying the sum-product-algorithm:

1. Define potentials for observed variables (in this case $D = d^*$ and $E = e^*$):
   a. $f_5(D) = \begin{cases} 1, & D = d^* \\ 0, & D \neq d^* \end{cases}$
   b. $f_6(E) = \begin{cases} 1, & E = e^* \\ 0, & E \neq e^* \end{cases}$
2. Select root (e.g. $f_5$), doesn't matter much.
3. collect (start at leaf node): **(see blackboard)**

# Message passing exercise




## sum-product

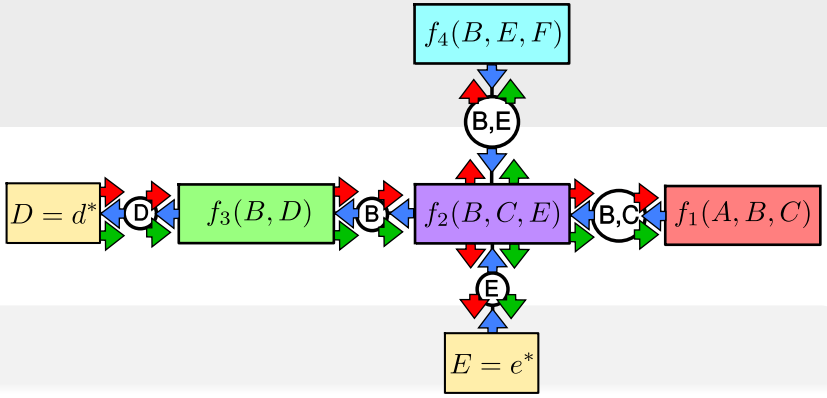
$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k)$$

Define the messages by applying the sum-product-algorithm:

3. collect (start at leaf node):
   a. 'branch on the right':
      i. $\mu_{f_1 \to B,C}(B,C) := \sum_a f_1(a,B,C) \overbrace{\cancel{\prod_{k \in \text{neigh.}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(X_k)}}^{\text{no other neighbors}}$
      ii. $\mu_{B,C \to f_2}(B,C) := \mu_{f_1 \to B,C}(B,C) = \sum_a f_1(a,B,C)$

# Message passing exercise




## sum-product

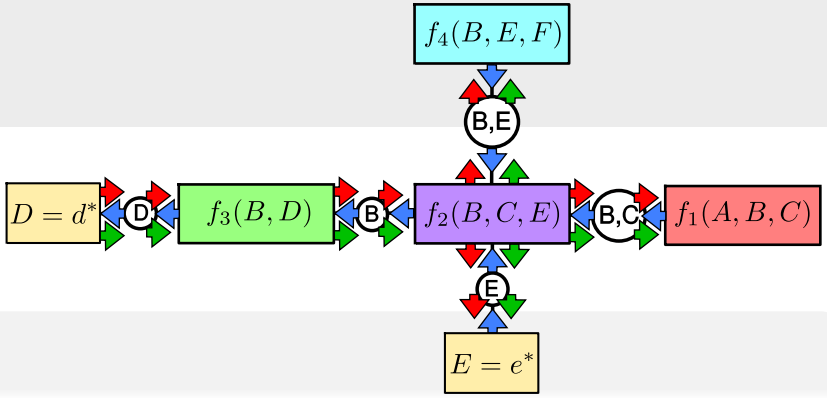
$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k)$$

Define the messages by applying the sum-product-algorithm:

3. collect (start at leaf node):
   b. 'top branch':
      i. $\mu_{f_4 \to B,E}(B,E) := \sum_f f_4(B,E,f)$
      ii. $\mu_{B,E \to f_2}(B,E) := \mu_{f_4 \to B,E}(B,E) = \sum_f f_4(B,E,f)$

# Message passing exercise




## sum-product

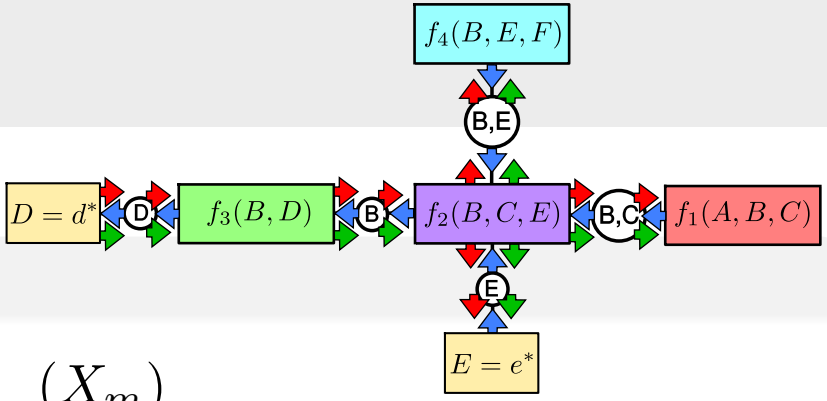
$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k)$$

Define the messages by applying the sum-product-algorithm:

3. collect (start at leaf node):
   c. 'bottom branch':
      i. $\mu_{f_6 \to E}(E) := f_6(E)$
      ii. $\mu_{E \to f_2}(E) := \mu_{f_6 \to E}(E) = f_6(E)$

# Message passing exercise




## sum-product

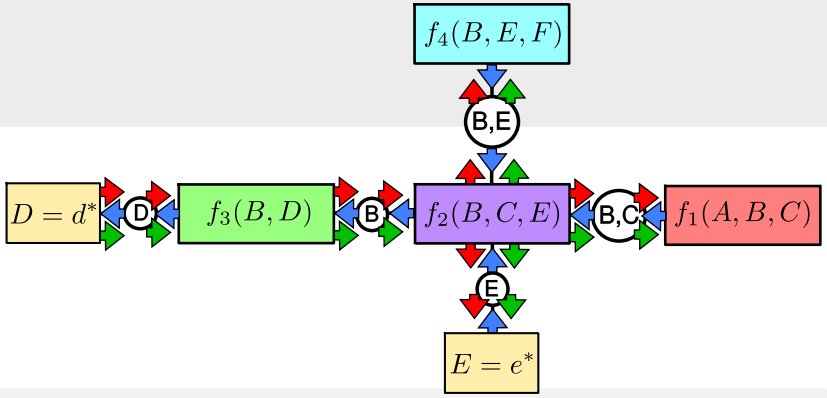
$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k)$$

Define the messages by applying the sum-product-algorithm:

3. collect (start at leaf node):
   d. 'left branch':
      i. $\mu_{f_2 \to B}(B) := \sum_{c,e} f_2(B,c,e) \cdot \mu_{E \to f_2}(e) \cdot \mu_{B,E \to f_2}(B,e) \cdot \mu_{B,C \to f_2}(B,c)$
      ii. $\mu_{B \to f_3}(B) := \mu_{f_2 \to B}(B)$
      iii. $\mu_{f_3 \to D}(D) := \sum_b f_3(b,D) \cdot \mu_{B \to f_3}(b)$
      iv. $\mu_{D \to f_5}(D) := \mu_{f_3 \to D}(D)$

# Message passing exercise




## sum-product

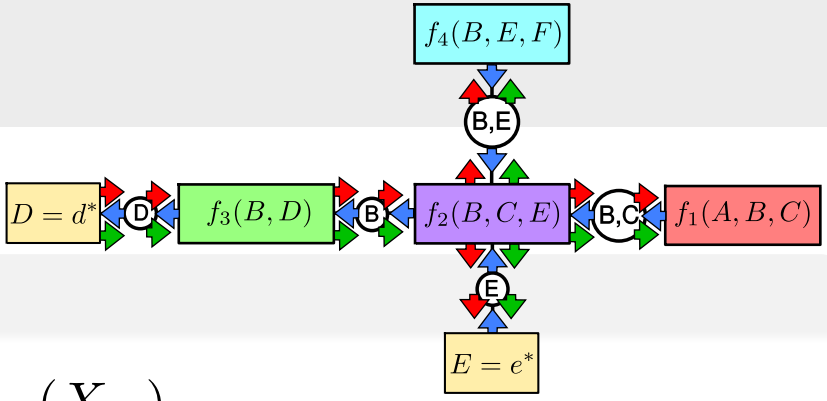
$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k)$$

Define the messages by applying the sum-product-algorithm:

4. distribute (start at root node):

# Message passing exercise




## sum-product

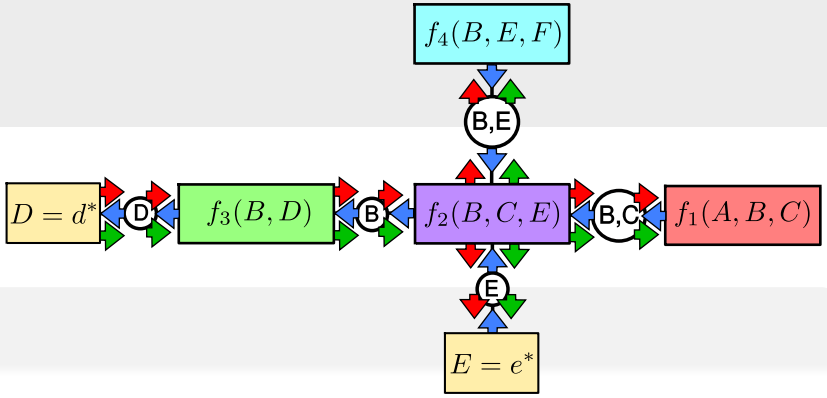
$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k)$$

Define the messages by applying the sum-product-algorithm:

4. distribute (start at root node):
   a. go right:
      i. $\mu_{f_5 \to D}(D) := f_5(D)$
      ii. $\mu_{D \to f_3}(D) := \mu_{f_5 \to D}(D) = f_5(D)$
      iii. $\mu_{f_3 \to B}(B) := \sum_d f_3(B, d) \cdot \mu_{D \to f_3}(d)$
      iv. $\mu_{B \to f_2}(B) := \mu_{f_3 \to B}(B)$

# Message passing exercise




## sum-product

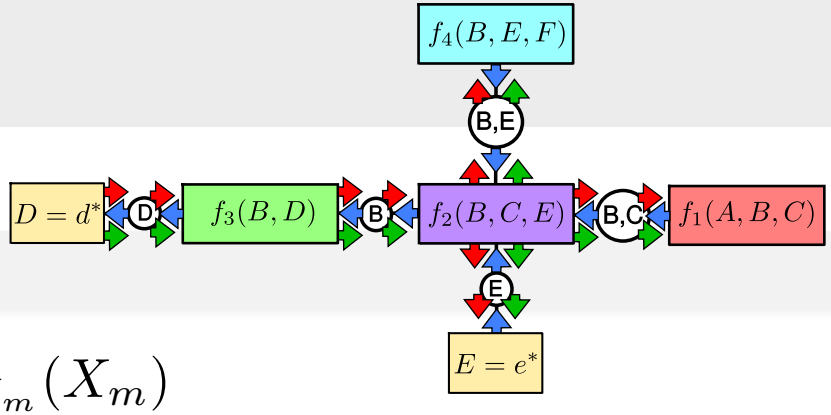
$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k)$$

Define the messages by applying the sum-product-algorithm:

4. distribute (start at root node):
   b. go further to the right:
      i. $\mu_{f_2 \to B,C}(B,C) := \sum_e f_2(B,C,e) \cdot \mu_{E \to f_2}(e) \cdot \mu_{B,E \to f_2}(B,e) \cdot \mu_{B \to f_2}(B)$
      ii. $\mu_{B,C \to f_1}(B,C) := \ldots$

# Message passing exercise




## sum-product

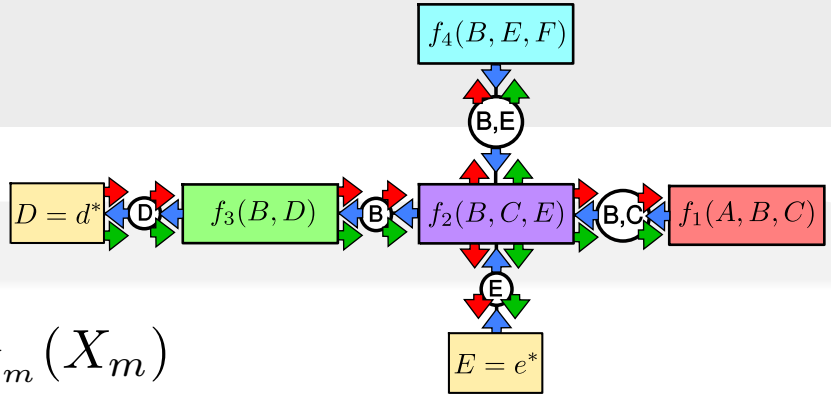
$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k)$$

Define the messages by applying the sum-product-algorithm:

4. distribute (start at root node):
   - b. go further to the right:
     - i. $\mu_{f_2 \to B,C}(B,C) := \sum_e f_2(B,C,e) \cdot \mu_{E \to f_2}(e) \cdot \mu_{B,E \to f_2}(B,e) \cdot \mu_{B \to f_2}(B)$
     - ii. $\mu_{B,C \to f_1}(B,C) := \ldots$
   - c. go up: ...
   - d. go down: ...

# Message passing exercise




## sum-product

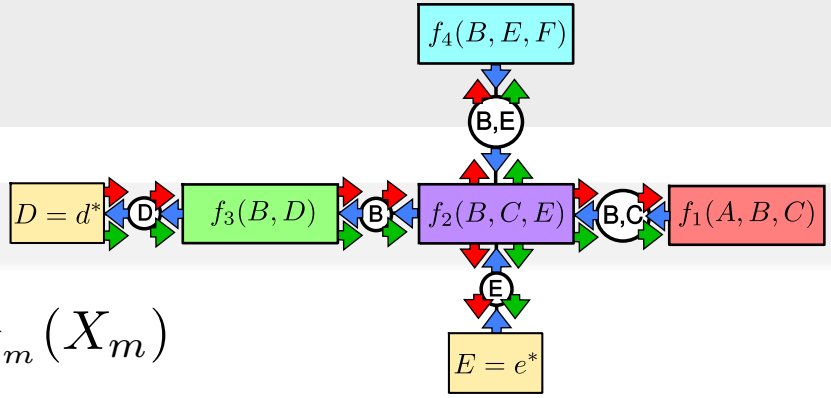
$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k)$$

Define the messages by applying the sum-product-algorithm:

4. distribute (start at root node):
5. calculate marginals:
   a. $$P(C, D = d^*, E = e^*) = \sum_b \mu_{f_1 \to BC}(B,C) \cdot \mu_{f_2 \to BC}(B,C)$$
   b. other queries you are interested in ...

# Message passing exercise




## sum-product

$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m)$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k)$$

Define the messages by applying the sum-product-algorithm:

4. distribute (start at root node):
6. normalize to compute conditional marginals:

$$P(C \,|\, D = d^*, E = e^*) = \alpha \underbrace{P(C, D = d^*, E = e^*)}_{\tilde{P}(C)},$$

$$\text{where } \alpha = \frac{1}{\sum_c \tilde{P}(c)}$$

# References I

Christopher M Bishop. *Pattern recognition and machine learning*. springer, 2006. URL http://users.isr.ist.utl.pt/~wurmd/Livros/school/Bishop%20-%20Pattern%20Recognition%20And%20Machine%20Learning%20-%20Springer%20%202006.pdf.

Daphne Koller and Nir Friedman. *Probabilistic graphical models: principles and techniques*. MIT press, 2009.

Stuart J Russell and Peter Norvig. *Artificial intelligence: a modern approach*. Malaysia; Pearson Education Limited,, 2016.